Guía de lógica de programación

Ejercicios resueltos en Java 21 · trazas · pseudocódigo · explicación línea por línea

Esta guía reúne los 19 proyectos del repositorio **Practicar logica**. Para cada uno se explica qué problema resuelve, cómo se razona la solución, qué hace cada línea, cuál es la salida y qué errores se corrigieron. También incluye un glosario extenso de los métodos y tipos que pediste: `stream`, `collect`, `Collectors`, `Collections`, `map`, `filter`, `frequency`, `<T> T`, `Pedido::cliente`, `groupingBy`, `Supplier`, `record`, `List`, `ArrayList`, `HashMap`, `Set` y `Class<?> c : classes`.

19 proyectos	**60** archivos Java	**Java 21** JDK	**0** dependencias externas

Qué vas a encontrar

- **Parte I:** los conceptos base explicados con ejemplos mínimos y el ejercicio donde se usan.
- **Parte II:** los 19 ejercicios, cada uno con objetivo, archivos, pasos, pseudocódigo, traza, código comentado, salida y errores corregidos.
- **Parte III:** tabla resumen de errores de compilación y lógica que se corrigieron.
- **Parte IV:** cómo compilar y ejecutar cada proyecto desde la terminal o desde VS Code.
- **Parte V:** checklist para seguir practicando y errores que conviene evitar.

Iracar Technologies · Octubre 2026

Contenido

Mapa del repositorio 4

Parte I — Conceptos base de Java 5

1. record: clases de datos inmutables . . . 5
2. List, ArrayList, Set y HashSet . . . 5
3. Map, HashMap y Map.Entry . . . 6
4. Genéricos: <T> T, Class<T> y Class<?> . . . 6
5. Optional: el valor puede no existir . . . 7
6. stream: la cadena de procesamiento . . . 7
7. filter, map, flatMap, mapToDouble, sorted, limit, average . . . 8
8. collect y Collectors: convertir el stream en una colección . . . 8
9. groupingBy: agrupar como una tabla dinámica . . . 9
10. Collections.frequency y la diferencia con Collectors . . . 10
11. Referencias a métodos: Pedido::cliente y compañía . . . 10
12. Supplier: una fábrica de valores perezosa . . . 10
13. El bucle Class<?> c : classes (for-each y reflexión) . . . 11
14. AtomicLong, ConcurrentHashMap y Thread.sleep . . . 11
15. Scanner, consola y cierre de System.in . . . 12
16. Otros métodos que aparecen en el código . . . 12
17. Resumen visual del flujo de un stream . . . 12

Parte II — Ejercicios resueltos 14

1. Sumar dos números (Two Sum) 15

2. Operaciones básicas: doble, triple y conversión de grados 17

3. Arreglos, entrada por consola y mediana de dos arreglos ordenados 19

4. Listas: filtrar, aplanar, duplicados y Top N 24

5. Contar la frecuencia de palabras 29

6. Agrupar personas por ciudad y contarlas 31

7. Precio promedio con filtros 33

8. Patrón Strategy: descuentos intercambiables 35

9. Inyección de dependencias sin Spring 38

10. Mini-repositorio en memoria 40

11. API de tareas en consola por capas 43

12. Sistema de pedidos con relaciones 48

13. Validación de emails: sin regex y con regex 53

14. Paginación manual (Pageable) 57

15. Cache con expiración (TTL) 60

16. Reintentos con backoff exponencial 63

17. Parseo manual de JSON simple 66

18. Lista enlazada: operaciones y algoritmos 69

19. Mini Spring: anotación y contenedor de beans 77

Parte III — Errores corregidos 81

Parte IV — Cómo compilar y ejecutar 82

Desde VS Code . . . 82

Desde PowerShell, todo el repositorio . . . 82

Desde PowerShell, un solo proyecto . . . 82

Clases con main . . . 82

Parte V — Checklist y siguientes pasos 84

Checklist por conceptos . . . 84

Errores que conviene evitar . . . 84

Siguientes ejercicios recomendados . . . 84

Cómo estudiar con este PDF . . . 85

Mapa del repositorio

Cada carpeta es un proyecto independiente con su propia carpeta `src`. Todos comparten el mismo directorio de salida `bin` configurado en `.vscode/settings.json`. La tabla siguiente sirve como índice rápido: qué problema resuelve cada proyecto y qué conceptos conviene repasar antes.

#	Proyecto	Problema	Conceptos clave
1	SumarDosNumeros	Encontrar dos índices que sumen un target	`HashMap`, complemento
2	Operaciones	Doble, triple y conversión °C -> °F	`Scanner`, métodos
3	Colecciones	Arreglos y mediana de dos arreglos ordenados	búsqueda binaria, `int[]`
4	Listas	Filtrar, aplanar, duplicados y Top N	`stream`, `flatMap`, `groupingBy`
5	Mapas	Frecuencia de palabras	`toMap`, merge
6	Colectores	Agrupar por ciudad y contar	`groupingBy`, `counting`
7	Promedio	Promedio condicional de productos	`filter`, `mapToDouble`, `average`
8	Descuento	Estrategias de descuento	interfaz, polimorfismo, Strategy
9	InyeccionDeDependencias	Notificador con sender intercambiable	inyección por constructor
10	Repositorio	CRUD en memoria de usuarios	`Map`, `Optional`, `AtomicLong`
11	ApiConsola	API de tareas por capas	capas, `Map`, `Optional`
12	Pedidos	Total, cliente top y producto top	records, `groupingBy`, `flatMap`
13	Validar	Validar emails sin/con regex	`String`, `Pattern`
14	Paginas	Paginación manual	genéricos, `subList`, `Math.ceil`
15	CacheProblema	Cache con TTL	genéricos, `ConcurrentHashMap`
16	RetryProblema	Reintentos con backoff	`Supplier`, `Thread.sleep`
17	ParseoProblema	Extraer un valor de JSON plano	`indexOf`, `substring`
18	Nodos	Lista enlazada y algoritmos	punteros, Floyd
19	MiniSpring	Contenedor de beans con anotaciones	reflexión, `Class<?>`

Los proyectos 1 a 7 son de lógica y streams; 8 a 12 son de diseño de clases y capas; 13 a 17 son de validación, paginación, cache, reintentos y parseo; 18 y 19 son de estructuras de datos y reflexión. Si estás empezando, hazlos en ese orden.

Parte I — Conceptos base de Java

Antes de entrar ejercicio por ejercicio, conviene tener claras las piezas que se repiten. En esta parte explico cada concepto que pediste: `stream`, `collect`, `Collectors`, `Collections`, `map`, `filter`, `frequency`, `<T> T`, `Pedido::cliente`, `groupingBy`, `Supplier`, `record`, `List`, `ArrayList`, `HashMap`, `Set` y el bucle `Class<?> c : classes`. Cada explicación incluye un ejemplo mínimo y el ejercicio real donde se usa.

1. record: clases de datos inmutables

Un `record` es una forma corta de declarar una clase que solo transporta datos. Java genera automáticamente el constructor, los métodos de acceso, `equals`, `hashCode` y `toString`. Los campos son `private final`, así que un record no se modifica: si necesitas cambiar un dato, creas otro objeto.

```
// Forma tradicional (muchas líneas)
public class Usuario {
    private final Long id;
    private final String nombre;
    public Usuario(Long id, String nombre) { this.id = id; this.nombre = nombre; }
    public Long id() { return id; }
    public String nombre() { return nombre; }
    // equals, hashCode, toString...
}

// Con record (una línea)
public record Usuario(Long id, String nombre) {}

Usuario u = new Usuario(1L, "Ana");
System.out.println(u.nombre());   // Ana  (no getName(), se llama igual que el campo)
System.out.println(u);            // Usuario[id=1, nombre=Ana]
```

Los métodos de acceso se llaman igual que los componentes: `u.id()`, `u.nombre()`, `p.cliente()`. No se usa `getId()`. Un record puede tener métodos normales, como `subtotal()` en `LineaPedido` o `total()` en `Pedido`, y también constructores compactos para validar.

```
record LineaPedido(Producto producto, int cantidad) {
    public double subtotal() {          // método extra permitido
        return producto.precio() * cantidad;
    }
}
```

Dónde se usa: `Usuario` (Repositorio), `Cliente`, `Producto`, `LineaPedido`, `Pedido` (Pedidos), `Tarea` (ApiConsola), `Entry` (Cache), `Producto` (Promedio), `Persona` (Colectores), `Venta` (TopN).

> **Ojo con la visibilidad**
> Un record declarado dentro de un archivo puede quedar con visibilidad de paquete. Si otra clase de otro paquete lo necesita, el record debe ser `public` y, normalmente, estar en su propio archivo con el mismo nombre. Ese fue uno de los problemas de Pedidos y ApiConsola.

2. List, ArrayList, Set y HashSet

`List` y `Set` son **interfaces** (contratos). `ArrayList` y `HashSet` son **implementaciones** concretas. Programar contra la interfaz (`List<String>`) permite cambiar la implementación sin tocar el resto del código.

Tipo	¿Ordena?	¿Permite duplicados?	Acceso	Uso típico
`List`	Sí, por índice	Sí	`get(i)`	Secuencias, resultados de streams
`ArrayList`	Sí	Sí	`get(i)` rápido	Lista dinámica que crece
`Set`	No garantizado	No	Sin índice	Evitar duplicados

Tipo	¿Ordena?	¿Permite duplicados?	Acceso	Uso típico
`HashSet`	No garantizado	No	Sin índice	Búsquedas rápidas `contains`

```
List<String> nombres = List.of("Ana", "Luis", "Ana"); // inmutable, permite duplicados
// nombres.add("Eva");  // UnsupportedOperationException

List<String> mutable = new ArrayList<>(nombres); // copia mutable
mutable.add("Eva");
mutable.remove("Luis");
System.out.println(mutable);        // [Ana, Ana, Eva]
System.out.println(mutable.get(0)); // Ana
System.out.println(mutable.size()); // 3

Set<String> unicos = new HashSet<>(nombres);
System.out.println(unicos);         // [Ana, Luis]  (sin duplicados, sin orden fijo)
```

Dónde se usa: `List<String>` en FiltrarTransformarListas; `new ArrayList<>(db.values())` en Repositorio y ApiConsola; `Set<Integer>` en NumerosRepetidos; `List<List<Integer>>` en EstructurasAnidadas; `List.of(...)` en casi todos los ejercicios.

List.of es inmutable
`List.of(...)` crea una lista que no se puede modificar. Si necesitas `add`, `remove` o `sort`, envuélvela en `new ArrayList<>(...)`. `subList` devuelve una vista de la lista original, no una copia.

3. Map, HashMap y Map.Entry

Un `Map<K, V>` guarda pares clave-valor. La clave es única; si repites una clave, el valor anterior se reemplaza. `HashMap` es la implementación más común: búsqueda, inserción y borrado en tiempo constante promedio, pero **no garantiza orden**.

```
Map<String, Integer> edades = new HashMap<>();
edades.put("Ana", 25);          // inserta
edades.put("Luis", 30);
edades.put("Ana", 26);          // reemplaza el valor de Ana

System.out.println(edades.get("Ana"));        // 26
System.out.println(edades.get("Eva"));        // null
System.out.println(edades.getOrDefault("Eva", 0)); // 0
System.out.println(edades.containsKey("Luis"));    // true
System.out.println(edades.keySet());          // [Ana, Luis]
System.out.println(edades.values());          // [26, 30]
System.out.println(edades.entrySet());        // [Ana=26, Luis=30]

for (Map.Entry<String, Integer> e : edades.entrySet()) {
    System.out.println(e.getKey() + " tiene " + e.getValue());
}
```

`Map.Entry<K, V>` representa una pareja dentro del mapa. `entrySet()` devuelve el conjunto de parejas, y eso es lo que permite ordenar un mapa por valor: `Map.Entry.comparingByValue()`. `HashMap` no es thread-safe; `ConcurrentHashMap` sí lo es y se usa en la cache.

Dónde se usa: `Map<Long, Usuario>` (Repositorio), `Map<Long, Tarea>` (ApiConsola), `Map<String, Integer>` (Mapas y TopN), `Map<String, Long>` (Colectores), `Map<String, Object>` (Paginación), `Map<Class<?>, Object>` (MiniSpring), `Map<K, Entry<V>>` (Cache).

4. Genéricos: <T> T, Class<T> y Class<?>

Los genéricos permiten escribir código que funciona con muchos tipos sin perder seguridad. `<T>` declara un parámetro de tipo; `T` es el nombre de ese tipo dentro del método o la clase. En `public static <T> List<T> obtenerPagina(List<T> lista, ...)`, la `T` significa: "el mismo tipo que venga en la lista, y devuelvo una lista

de ese tipo".

```
// Sin genéricos: habría que hacer una versión para Integer, otra para String...
public static List<Integer> paginaEnteros(List<Integer> l, int p, int s) { ... }

// Con genéricos: una sola versión para cualquier T
public static <T> List<T> obtenerPagina(List<T> lista, int pagina, int tamano) {
    return lista.subList(0, tamano);
}

List<Integer> enteros = obtenerPagina(List.of(1, 2, 3), 1, 2); // T = Integer
List<String> textos  = obtenerPagina(List.of("a", "b"), 1, 2); // T = String
```

`Class<T>` es el objeto que representa al tipo `T` en tiempo de ejecución. `Class<?>` es "una clase de cualquier tipo"; el comodín `?` se usa cuando no necesitas el tipo exacto. En MiniSpring, `Map<Class<?>, Object>` guarda beans de distintos tipos, y `getBean(Class<T> tipo)` devuelve `T` usando `tipo.cast(objeto)`.

```
public <T> T getBean(Class<T> tipo) {
    Object bean = beans.get(tipo);
    return tipo.cast(bean);   // convierte Object a T de forma segura
}

UserService us = ctx.getBean(UserService.class); // T = UserService
```

Dónde se usa: `PaginacionManualV1` y `V2` (`<T> List<T>`, `<T> Map<String, Object>`), `RetryBackoff` (`<T> T`), `MiniContenedor` (`Map<Class<?>, Object>`, `<T> T getBean`), `CacheExperiracion<K, V>`.

5. Optional: el valor puede no existir

`Optional<T>` es una caja que puede contener un valor o estar vacía. Se usa para evitar `null` y obligar a decidir qué hacer cuando no hay resultado. `Optional.ofNullable(x)` crea la caja; `isPresent()` dice si hay valor; `ifPresent(...)` ejecuta algo si lo hay; `orElse(valor)` devuelve un valor por defecto; `orElseThrow(...)` lanza una excepción si está vacío.

```
Optional<Usuario> encontrado = repo.findById(2L);

encontrado.ifPresent(u -> System.out.println("Encontrado: " + u));

Usuario u = repo.findById(2L).orElseThrow(() -> new RuntimeException("No existe"));
String nombre = repo.findById(2L).map(Usuario::nombre).orElse("Anónimo");
```

Dónde se usa: `UsuarioRepository.findById`, `ApiRepository.findById`, `ApiService.completar`, `SumaPromedio` (`average()` devuelve `OptionalDouble`), `PedidosRepository` (`max(...).map(...).orElse`).

6. stream: la cadena de procesamiento

Un `stream` es una secuencia de datos que se procesa por etapas. No guarda los datos: los toma de una fuente (lista, arreglo, mapa), los pasa por operaciones intermedias y termina con una operación terminal. Las intermedias son **perezosas**: no se ejecutan hasta que llega la terminal.

Etapa	Ejemplos	¿Devuelve otro stream?
Fuente	`lista.stream()`, `Arrays.stream(...)`, `map.entrySet().stream()`	Sí
Intermedia	`filter`, `map`, `flatMap`, `sorted`, `limit`, `distinct`	Sí
Terminal	`collect`, `toList`, `forEach`, `sum`, `average`, `max`, `count`	No

```
List<String> nombres = List.of("Ana", "pedro", "Antonio", "Luis", "alberto");

List<String> resultado = nombres.stream()          // fuente
    .filter(n -> n.toLowerCase().startsWith("a"))  // intermedia: deja pasar algunos
    .map(String::toUpperCase)                       // intermedia: transforma cada uno
    .sorted()                                       // intermedia: ordena
    .toList();                                      // terminal: materializa

// resultado = [ALBERTO, ANA, ANTONIO]
```

Un stream se recorre una sola vez. Si lo consumes y luego intentas usarlo otra vez, obtienes `IllegalStateException`. Tampoco debes modificar la lista original dentro del stream.

> **Regla práctica**
> Piensa en el stream como una tubería: `fuente -> filtro -> transformación -> orden -> recolección`. Cada etapa recibe lo que produjo la anterior.

7. filter, map, flatMap, mapToDouble, sorted, limit, average

Método	Qué hace	Ejemplo	Ejercicio
`filter`	Deja pasar solo los elementos que cumplen un `Predicate`	`filter(n -> n % 2 == 0)`	Listas, Promedio, Validar
`map`	Transforma cada elemento 1 a 1	`map(String::toUpperCase)`	FiltrarTransformar
`flatMap`	Transforma cada elemento en un stream y aplana el resultado	`flatMap(List::stream)`	EstructurasAnidadas, Pedidos
`mapToDouble`	Convierte a un `DoubleStream` para operaciones numéricas	`mapToDouble(Producto::precio)`	Promedio, Pedidos
`sorted`	Ordena de forma natural o con un `Comparator`	`sorted(Comparator.reverseOrder())`	Listas
`limit`	Corta el stream a los primeros N elementos	`limit(3)`	TopN
`average`	Promedio de un stream numérico; devuelve `OptionalDouble`	`average().orElse(0.0)`	Promedio
`sum`	Suma de un stream numérico	`mapToDouble(...).sum()`	Pedido.total()
`max`	Máximo según un comparador; devuelve `Optional`	`max(Map.Entry.comparingByValue())`	TopN, Pedidos

```
// filter: se queda con los pares
List.of(1, 2, 3, 4).stream().filter(n -> n % 2 == 0).toList(); // [2, 4]

// map: convierte cada número en su cuadrado
List.of(1, 2, 3).stream().map(n -> n * n).toList();            // [1, 4, 9]

// flatMap: List<List<Integer>> -> List<Integer>
List<List<Integer>> datos = List.of(List.of(1, 2), List.of(3, 4));
datos.stream().flatMap(List::stream).toList();                 // [1, 2, 3, 4]

// mapToDouble + average
List.of(10, 20, 30).stream().mapToDouble(n -> n).average();    // OptionalDouble[20.0]

// sorted + limit
List.of(5, 1, 4, 2, 3).stream().sorted().limit(3).toList();    // [1, 2, 3]
```

`map` y `flatMap` se confunden. `map` conserva la estructura (una lista de listas sigue siendo una lista de listas); `flatMap` la aplana (una lista de listas se convierte en una lista de elementos).

8. collect y Collectors: convertir el stream en una colección

`collect` es una operación terminal que recibe un `Collector`. `Collectors` es una clase de fábrica con métodos estáticos que crean esos colectores. Los más usados:

Colector	Resultado	Ejemplo
`toList()`	`List`	`stream.collect(Collectors.toList())`
`toSet()`	`Set` (sin duplicados)	`stream.collect(Collectors.toSet())`
`toMap(k, v, merge)`	`Map`	`toMap(w -> w, w -> 1, Integer::sum)`
`groupingBy(clasificador)`	`Map<K, List<V>>`	`groupingBy(Persona::ciudad)`
`groupingBy(clasificador, downstream)`	`Map<K, R>`	`groupingBy(Persona::ciudad, counting())`
`counting()`	`Long` con la cantidad	`groupingBy(..., counting())`
`summingInt(...)`	`Integer` con la suma	`groupingBy(..., summingInt(Venta::cantidad))`
`summingDouble(...)`	`Double` con la suma	`groupingBy(..., summingDouble(Pedido::total))`
`joining(...)`	`String` concatenado	`map(...).collect(joining(", "))`

```
List<String> palabras = List.of("spring", "boot", "spring");

// toList: lista
List<String> lista = palabras.stream().collect(Collectors.toList());

// toSet: sin duplicados
Set<String> conjunto = palabras.stream().collect(Collectors.toSet()); // [spring, boot]

// toMap: clave -> valor; el tercer argumento resuelve claves repetidas
Map<String, Integer> frecuencia = palabras.stream()
    .collect(Collectors.toMap(w -> w, w -> 1, Integer::sum)); // {spring=2, boot=1}

// groupingBy + counting
Map<Integer, Long> porLongitud = palabras.stream()
    .collect(Collectors.groupingBy(String::length, Collectors.counting()));
```

`toMap` sin el tercer argumento lanza `IllegalStateException` si hay una clave repetida. Por eso en `RepetirPalabras` se pasa `Integer::sum`: si la palabra ya existe, suma 1 al valor anterior.

9. groupingBy: agrupar como una tabla dinámica

`groupingBy` recorre el stream y crea un `Map`. La función clasificadora decide la clave de cada elemento; el colector descendente decide qué se guarda como valor. Si no se indica descendente, el valor es una `List` con todos los elementos del grupo.

```
record Persona(String nombre, String ciudad) {}
List<Persona> personas = List.of(
    new Persona("Ana", "Madrid"),
    new Persona("Luis", "Madrid"),
    new Persona("Eva", "Barcelona")
);

// Sin descendente: Map<String, List<Persona>>
Map<String, List<Persona>> porCiudad = personas.stream()
    .collect(Collectors.groupingBy(Persona::ciudad));

// Con counting: Map<String, Long>
Map<String, Long> cuantos = personas.stream()
    .collect(Collectors.groupingBy(Persona::ciudad, Collectors.counting()));
// {Madrid=2, Barcelona=1}
```

Dónde se usa: `AgrupacionColectores` (ciudad -> cantidad), `TopN` (producto -> unidades), `PedidosRepository` (cliente -> gasto; producto -> unidades), `RepetirPalabras` (palabra -> veces).

10. Collections.frequency y la diferencia con Collectors

`java.util.Collections` es una clase de utilidades para colecciones. `Collections.frequency(coleccion, elemento)` cuenta cuántas veces aparece un elemento. No es un stream ni un colector: recorre la colección completa cada vez que lo llamas.

```
List<Integer> nums = List.of(1, 2, 3, 2, 4, 5, 1, 6);

int veces = Collections.frequency(nums, 2); // 2

// Duplicados: deja pasar los que aparecen más de una vez y los mete en un Set
Set<Integer> duplicados = nums.stream()
    .filter(n -> Collections.frequency(nums, n) > 1)
    .collect(Collectors.toSet()); // [1, 2]
```

Cuidado con el costo
`Collections.frequency` recorre toda la lista. Usarlo dentro de un `filter` para cada elemento cuesta $O(n^2)$. Para listas grandes conviene un `HashMap` que cuente en una sola pasada, o `Collectors.groupingBy(n -> n, Collectors.counting())`.

No confundas: `Collection` es la interfaz padre de `List`/`Set`; `Collections` es la clase de utilidades; `Collectors` es la clase de fábrica de colectores para streams. Los tres nombres se parecen pero cumplen papeles distintos.

11. Referencias a métodos: Pedido::cliente y compañía

Una referencia a método (`::`) es una forma compacta de escribir una lambda que solo llama a un método. `Pedido::cliente` equivale a `p -> p.cliente()`. No ejecuta el método en ese momento: lo usa como función cuando el stream la necesite.

Referencia	Lambda equivalente	Qué hace
`Pedido::cliente`	`p -> p.cliente()`	Extrae el cliente de cada pedido
`String::toUpperCase`	`s -> s.toUpperCase()`	Transforma cada texto
`LineaPedido::subtotal`	`l -> l.subtotal()`	Calcula el subtotal de cada línea
`Map.Entry::getKey`	`e -> e.getKey()`	Extrae la clave de una entrada
`System.out::println`	`x -> System.out.println(x)`	Imprime cada elemento
`beans::get`	`tipo -> beans.get(tipo)`	Busca un bean por su clase

```
// Estas dos formas son equivalentes:
Map<Cliente, Double> a = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::cliente, Collectors.summingDouble(Pedido::total)));

Map<Cliente, Double> b = pedidos.stream()
    .collect(Collectors.groupingBy(p -> p.cliente(), Collectors.summingDouble(p -> p.total())));
```

Hay cuatro sabores de `::`: método estático (`Integer::parseInt`), método de un objeto concreto (`System.out::println`), método de un objeto arbitrario del tipo (`String::toUpperCase`) y constructor (`ArrayList::new`). `Pedido::cliente` es del tercer tipo.

12. Supplier: una fábrica de valores perezosa

`Supplier<T>` es una interfaz funcional con un solo método: `T get()`. No recibe argumentos y devuelve un valor. Se usa cuando quieres pasar una acción que se ejecutará después, o cuando necesitas crear un valor solo si hace falta.

```
Supplier<String> saludo = () -> "Hola";
System.out.println(saludo.get()); // Hola

// orElseThrow recibe un Supplier<RuntimeException>: solo crea la excepción si está vacío
Usuario u = repo.findById(id)
                .orElseThrow(() -> new RuntimeException("No existe"));

// RetryBackoff recibe un Supplier<T>: la tarea se ejecuta (y se reintenta) dentro del método
String resultado = ejecutarConReintentos(() -> conectar(), 3);
```

Interfaz	Firma	Recibe	Devuelve	Uso
`Supplier<T>`	`T get()`	Nada	`T`	Crear o producir un valor
`Consumer<T>`	`void accept(T)`	`T`	Nada	Consumir / imprimir
`Function<T,R>`	`R apply(T)`	`T`	`R`	Transformar
`Predicate<T>`	`boolean test(T)`	`T`	`boolean`	Filtrar
`Runnable`	`void run()`	Nada	Nada	Ejecutar una acción

Dónde se usa: `RetryBackoff.ejecutarConReintentos(Supplier<T>, int)` y `Optional.orElseThrow(Supplier<? extends X>)` en `ApiService`.

13. El bucle Class<?> c : classes (for-each y reflexión)

`for (Class<?> c : classes)` es un for-each: recorre la colección `classes` y en cada vuelta asigna a `c` un elemento. `Class<?>` significa "un objeto que representa una clase, de cualquier tipo". Con `c` puedes preguntar por anotaciones, constructores y métodos (reflexión).

```
List<Class<?>> classes = List.of(EmailService.class, UserService.class);

for (Class<?> c : classes) {
    System.out.println(c.getName());                // miniSpring.Service.EmailService
    System.out.println(c.isAnnotationPresent(MiServicio.class));

    Constructor<?>[] constructores = c.getConstructors();
    if (constructores[0].getParameterCount() == 0) {
        Object instancia = c.getDeclaredConstructor().newInstance();
    }
}
```

Equivale a usar un iterador, pero es más legible. `c.getConstructors()` devuelve solo constructores públicos; `getDeclaredConstructor()` devuelve el constructor sin argumentos (público o no) y `newInstance()` crea el objeto. En MiniSpring, el contenedor usa estas llamadas para instanciar los beans automáticamente.

14. AtomicLong, ConcurrentHashMap y Thread.sleep

`AtomicLong` es un contador thread-safe. `getAndIncrement()` devuelve el valor actual y luego suma 1, todo en una operación atómica. `ConcurrentHashMap` es un mapa thread-safe. Se usan cuando varios hilos pueden tocar el mismo dato.

```
AtomicLong seq = new AtomicLong(1);
Long id1 = seq.getAndIncrement(); // 1
Long id2 = seq.getAndIncrement(); // 2

Map<String, String> cache = new ConcurrentHashMap<>();
cache.put("clave", "valor");
cache.get("clave");
```

`Thread.sleep(milisegundos)` pausa el hilo actual. Se usa en `RetryBackoff` para esperar entre reintentos y en la demo de la cache para dejar pasar el TTL. Puede lanzar `InterruptedException`, por eso se captura y se restaura el flag con `Thread.currentThread().interrupt()`.

15. Scanner, consola y cierre de System.in

`Scanner` lee de `System.in`. `nextInt()` lee un entero, `nextDouble()` un decimal, `next()` una palabra y `nextLine()` una línea completa. Mezclar `nextInt()` con `nextLine()` es una fuente clásica de errores: `nextInt()` deja el salto de línea pendiente y el siguiente `nextLine()` lee vacío.

```
Scanner sc = new Scanner(System.in);
int opcion = sc.nextInt();
sc.nextLine();                // consume el salto de línea pendiente
String titulo = sc.nextLine(); // ahora sí lee la línea completa

// Si cierras el Scanner, cierras System.in para toda la aplicación.
// Por eso es mejor tener un solo Scanner y cerrarlo al final.
```

Dónde se usa: `Colecciones`, `Operaciones`, `ApiController`. En los dos primeros se corrigió el cierre del Scanner dentro de un bucle, que provocaba `NoSuchElementException` o `IllegalStateException`.

16. Otros métodos que aparecen en el código

Método / clase	Qué hace	Dónde aparece
`String.toLowerCase()`	Pasa a minúsculas	FiltrarTransformarListas
`String.startsWith("a")`	Comprueba el prefijo	FiltrarTransformarListas
`String.split("\\s+")`	Parte por uno o más espacios	RepetirPalabras
`String.indexOf / lastIndexOf`	Busca posición de un carácter o texto	ParseoJson, ValidarEmail
`String.substring(a, b)`	Extrae desde a hasta antes de b	ParseoJson, ValidarEmail
`String.isBlank()`	True si está vacío o solo espacios	ApiService.crear
`String.equalsIgnoreCase`	Compara sin distinguir mayúsculas	SumaPromedio
`Math.min / max`	Mínimo y máximo	Paginación, mediana
`Math.ceil`	Redondea hacia arriba	Paginación (total de páginas)
`Math.pow(2, i)`	Potencia; base del backoff exponencial	RetryBackoff
`Integer.MIN_VALUE / MAX_VALUE`	Centinelas para los extremos	Mediana
`System.currentTimeMillis()`	Milisegundos desde 1970	Cache (TTL)
`List.of / Map.of`	Colecciones inmutables	Casi todos
`LinkedHashMap`	Mapa que conserva el orden de inserción	PaginacionManualV2
`Comparator.reverseOrder()`	Orden inverso natural	EstructurasAnidadas
`Arrays.toString(...)`	Imprime un arreglo legible	Colecciones, SumarDosNumeros

17. Resumen visual del flujo de un stream

```
fuente                intermedias                        terminal
------                -----------                        --------
lista.stream()  ->  filter(...)  ->  map(...)  ->  sorted()  ->  toList()
                                                                 collect(...)
                                                                 forEach(...)
                                                                 sum() / average()
```

Cada ejercicio de la Parte II se puede leer con esta misma idea: identificar la fuente, las transformaciones y la operación final. Cuando algo no cuadra, vuelve a esta tabla y pregúntate qué tipo entra y qué tipo sale en cada etapa.

Parte II — Ejercicios resueltos

Cada capítulo sigue la misma estructura: qué pide el ejercicio, qué archivos intervienen, los pasos mentales, un pseudocódigo o traza, el código completo comentado, la salida esperada y los errores que se corrigieron. Los ejercicios están ordenados de menor a mayor complejidad, pero puedes saltar al que te interese usando el índice.

1. Sumar dos números (Two Sum)

Dado un arreglo de enteros y un objetivo `target`, encontrar los índices de dos números que sumen exactamente `target`. Se asume que existe exactamente una solución y que no se puede usar el mismo elemento dos veces. Ejemplo: `nums = [2, 7, 11, 15]`, `target = 9` -> `[0, 1]` porque `2 + 7 = 9`.

Archivos

Archivo	Qué contiene
SumarDosNumeros/src/sumardosnumeros/SumarDosNumeros.java	Solución con `HashMap` y `main` de prueba.

Paso a paso

1. Recorrer el arreglo una sola vez con un índice `i`.
2. Para cada número `nums[i]`, calcular qué número falta para llegar al objetivo: `complemento = target - nums[i]`.
3. Preguntar al mapa si ya vimos ese complemento antes. El mapa guarda `número -> índice`.
4. Si existe, devolver los dos índices: el que está guardado y el actual `i`.
5. Si no existe, guardar el número actual con su índice y continuar.
6. Si se termina el recorrido sin encontrarlo, devolver un arreglo vacío.

Pseudocódigo / traza

```
para i desde 0 hasta nums.length - 1:
    complemento = target - nums[i]
    si mapa.contiene(complemento):
        regresar [mapa[complemento], i]
    mapa[nums[i]] = i
regresar []

Traza con nums = [2, 7, 11, 15], target = 9:
i=0, num=2,  complemento=7  -> mapa vacío -> guardar 2 -> 0
i=1, num=7,  complemento=2  -> mapa contiene 2 -> regresar [0, 1]
```

Código completo

```java
package sumardosnumeros;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public class SumarDosNumeros {

    public static int[] twoSum(int[] nums, int target){
        // Clave = número, valor = índice donde aparece.
        Map<Integer,Integer> mapa = new HashMap<>();

        // Recorremos el arreglo una sola vez: O(n).
        for(int i = 0; i < nums.length; i++){
            // ¿Qué número necesito para llegar al target?
            int complemento = target - nums[i];

            // Si ya vi el complemento, la respuesta son ambos índices.
            if(mapa.containsKey(complemento)){
                return new int[]{mapa.get(complemento), i};
            }

            // Si no, guardo el número actual y su índice para futuras vueltas.
            mapa.put(nums[i], i);
        }

        // Según el enunciado nunca se llega aquí.
        return new int[]{};
    }

    public static void main(String[] args) {
        int[] nums = {2, 7, 11, 15};
        int target = 9;
        System.out.println(Arrays.toString(twoSum(nums, target))); // [0, 1]
    }
}
```

Salida esperada

```
[0, 1]
```

¿Por qué un mapa y no dos bucles?
Dos bucles anidados cuestan O(n^2). El mapa convierte la búsqueda del complemento en O(1) promedio, así que el algoritmo completo es O(n) en tiempo y O(n) en memoria.

Conceptos usados

`int[]`, `HashMap`, `containsKey`, `get`, `put`, `for`, `return`, `Arrays.toString`.

2. Operaciones básicas: doble, triple y conversión de grados

Practicar entrada por consola, operaciones aritméticas y métodos. `multiplos()` pide un entero y muestra su doble y su triple. `grados()` convierte grados Celsius a Fahrenheit con la fórmula `F = 32 + (9 * C / 5)`. El `main` pide los grados y llama al método privado.

Archivos

Archivo	Qué contiene
Operaciones/src/operaciones/Operaciones.java	Métodos `multiplos`, `grados` y `main`.

Paso a paso

1. Crear un `Scanner` para leer de la consola.
2. En `multiplos`, leer un entero y calcular `2 * numero` y `3 * numero`.
3. En `grados`, recibir un `double` y aplicar la fórmula de conversión.
4. En `main`, crear un objeto `Operaciones` para poder llamar a métodos de instancia.
5. Leer los grados, llamar a `grados(...)` e imprimir el resultado.
6. Cerrar el `Scanner` al final.

Pseudocódigo / traza

```
multiplos():
    leer numero
    doble = 2 * numero
    triple = 3 * numero
    imprimir doble, triple

grados(gradC):
    gradF = 32 + (9 * gradC / 5)
    imprimir gradF
    regresar gradF

main:
    crear objeto Operaciones
    leer gradC
    objeto.grados(gradC)

Traza: gradC = 25
gradF = 32 + (9 * 25 / 5) = 32 + 45 = 77.0
```

Código completo

```java
package operaciones;

import java.util.Scanner;

public class Operaciones {

    // Función para múltiplos de un número.
    public void multiplos(){
        Scanner sc = new Scanner(System.in);
        int numero;

        System.out.println("Introduce un número entero: ");
        numero = sc.nextInt();
        sc.close();

        double dob = 2 * numero;
        double tri = 3 * numero;

        System.out.println("El doble del número " + numero + " -> " + dob);
        System.out.println("El triple del número " + numero + " -> " + tri);
    }

    // Función para conversión de °C a °F.
    private double grados(double gradC){
        double gradF;
        gradF = 32 + (9 * gradC / 5);
        System.out.println("La conversión de los " + gradC + "°C = " + gradF + "°F");
        return gradF;
    }

    public static void main(String[] args) {
        Operaciones gd = new Operaciones();

        // gd.multiplos(); // descomenta para probar la otra opción

        Scanner sc = new Scanner(System.in);
        System.out.println("Ingresa los grados centígrados: ");
        double gradC;
```

```java
        gradC = sc.nextDouble();
        sc.close();
        gd.grados(gradC);
    }
}
```

Salida esperada

```
Ingresa los grados centígrados:
25
La conversión de los 25.0°C = 77.0°F
```

Detalle de la división
`gradC` es `double`, así que `9 * gradC / 5` se calcula con decimales. Si `gradC` fuera `int`, `9 * 25 / 5` también daría 45, pero en otros valores se perderían decimales por la división entera.

Conceptos usados

`Scanner`, `nextInt`, `nextDouble`, `double`, `método de instancia`, `private`, `return`.

3. Arreglos, entrada por consola y mediana de dos arreglos ordenados

Este proyecto tiene tres piezas: `Colecciones` (menú con `Scanner`), `metodosDeArreglos` (leer un arreglo de tamaño variable) y `findMedianSortedArrays` (calcular la mediana de dos arreglos **ya ordenados** en tiempo O(log(min(m, n)))).

Archivos

Archivo	Qué contiene
Colecciones/src/colecciones/Colecciones.java	Menú, arreglo fijo de 5 números y llamada a la mediana.
Colecciones/src/colecciones/metodosDeArreglos.java	Leer tamaño y elementos de un arreglo.
Colecciones/src/colecciones/findMedianSortedArrays.java	Búsqueda binaria sobre particiones.

Paso a paso

1. El `main` crea **un solo** `Scanner` y lo reutiliza en todas las opciones del menú.
2. La opción 1 lee 5 números en un arreglo fijo y los imprime con un `for` y con `Arrays.toString`.
3. La opción 2 pide el tamaño, crea `new int[tamano]`, lee cada posición y la imprime.
4. La opción 3 llama a `medianSorted(nums1, nums2)` con dos arreglos ordenados.
5. La opción 4 termina el bucle y cierra el `Scanner` una sola vez.

Pseudocódigo / traza

```
medianSorted(nums1, nums2):
    si nums1 es más grande que nums2:
        regresar medianSorted(nums2, nums1)   // intercambiar para buscar en el más pequeño

    m = tamaño de nums1, n = tamaño de nums2
    low = 0, high = m

    mientras low <= high:
        partitionX = (low + high) / 2
        partitionY = (m + n + 1) / 2 - partitionX

        maxLeftX  = nums1[partitionX - 1] o -infinito si partitionX = 0
        minRightX = nums1[partitionX]     o +infinito si partitionX = m
        maxLeftY  = nums2[partitionY - 1] o -infinito si partitionY = 0
        minRightY = nums2[partitionY]     o +infinito si partitionY = n

        si maxLeftX <= minRightY y maxLeftY <= minRightX:
            si (m+n) es par:
                regresar (max(maxLeftX, maxLeftY) + min(minRightX, minRightY)) / 2
            si no:
                regresar max(maxLeftX, maxLeftY)
        si maxLeftX > minRightY:
            high = partitionX - 1
        si no:
            low = partitionX + 1
```

findMedianSortedArrays.java

```java
package colecciones;

public class findMedianSortedArrays {

    public double medianSorted(int[] nums1, int[] nums2){
        // Aseguramos que nums1 sea el más pequeño (búsqueda binaria más eficiente).
        if(nums1.length > nums2.length){
            return medianSorted(nums2, nums1);
        }

        int m = nums1.length;
        int n = nums2.length;
        int low = 0, high = m;

        while(low <= high){
            int partitionX = (low + high) / 2;

            // La partición de nums2 se calcula para que la mitad izquierda
            // tenga (m+n+1)/2 elementos en total.
            int partitionY = (m + n + 1) / 2 - partitionX;

            int maxLeftX = (partitionX == 0) ? Integer.MIN_VALUE : nums1[partitionX - 1];
            int minRightX = (partitionX == m) ? Integer.MAX_VALUE : nums1[partitionX];

            int maxLeftY = (partitionY == 0) ? Integer.MIN_VALUE : nums2[partitionY - 1];
            int minRightY = (partitionY == n) ? Integer.MAX_VALUE : nums2[partitionY];

            // ¿La partición es correcta? Todo lo de la izquierda <= todo lo de la derecha.
            if(maxLeftX <= minRightY && maxLeftY <= minRightX){
                if((m + n) % 2 == 0){
                    return ((double) Math.max(maxLeftX, maxLeftY)
                            + Math.min(minRightX, minRightY)) / 2;
                } else {
                    return (double) Math.max(maxLeftX, maxLeftY);
                }
            } else if (maxLeftX > minRightY){
                high = partitionX - 1; // nos pasamos: mover a la izquierda
            } else {
                low = partitionX + 1;  // nos quedamos cortos: mover a la derecha
            }
        }
        throw new IllegalArgumentException("Los arreglos no están ordenados.");
    }
}
```

Colecciones.java (menú y correcciones)

```
package colecciones;

import java.util.Arrays;
import java.util.Scanner;

public class Colecciones {

    public static void arregloDefinido(Scanner sc){
        int[] nums = new int[5];

        for(int i = 0; i < nums.length; i++){
            System.out.println("Ingresa el número " + (i + 1) + ":");
            nums[i] = sc.nextInt();
        }

        System.out.println("Impresión con for:");
        for(int i = 0; i < nums.length; i++){
            System.out.println(nums[i]);
        }

        System.out.println("Impresión con Arrays.toString:");
        System.out.println(Arrays.toString(nums));
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // un solo Scanner para todo
        int opcion = 0;

        // El algoritmo de mediana exige arreglos ordenados.
        int[] nums1 = {2, 3, 5, 6};
        int[] nums2 = {3, 6, 9, 10};

        findMedianSortedArrays arr = new findMedianSortedArrays();
        metodosDeArreglos met = new metodosDeArreglos();

        while(opcion != 4){
            System.out.println("\nElige una opción:\n1. Arreglo definido (5 números)\n"
                    + "2. Ingresar el tamaño del arreglo\n3. Media de dos arreglos.\n4. Salir");
            opcion = sc.nextInt();

            switch(opcion){
                case 1: arregloDefinido(sc); break;
                case 2: met.arregloIngresado(sc); break;
                case 3: System.out.println("La mediana es: " + arr.medianSorted(nums1, nums2));
                    break;
                case 4: System.out.println("Hasta luego."); break;
                default: System.out.println("Ingresa un valor válido.");
            }
        }
        sc.close();
    }
}
```

metodosDeArreglos.java

```java
package colecciones;

import java.util.Arrays;
import java.util.Scanner;

public class metodosDeArreglos {

    public void arregloIngresado(Scanner sc){
        System.out.println("Ingresa el tamaño del arreglo:");
        int tamano = sc.nextInt();

        int[] nums = new int[tamano];

        for(int i = 0; i < nums.length; i++){
            System.out.println("Ingresa el número " + (i + 1) + ":");
            nums[i] = sc.nextInt();
        }

        System.out.println("Los números del arreglo son:");
        for(int i = 0; i < nums.length; i++){
            System.out.println(nums[i]);
        }

        System.out.println("Otra forma de imprimir el arreglo es la siguiente:");
        System.out.println(Arrays.toString(nums));
    }
}
```

Salida esperada

```
Elige una opción:
1. Arreglo definido (5 números)
2. Ingresar el tamaño del arreglo
3. Media de dos arreglos.
4. Salir
3
La mediana es: 5.5

Elige una opción:
...
4
Hasta luego.
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
Recursión infinita	`if (nums1.length > nums2.length) return medianSorted(nums1, nums2);` volvía a llamar con los mismos argumentos.	Se intercambiaron los argumentos: `return medianSorted(nums2, nums1);`.
Partición Y incorrecta	Faltaba restar `partitionX`; la mitad izquierda no tenía el tamaño correcto.	`int partitionY = (m + n + 1) / 2 - partitionX;`.
Scanner cerrado dentro del bucle	Al cerrar el `Scanner` se cerraba `System.in` y la siguiente lectura fallaba.	Un solo `Scanner` en `main`, cerrado al final; los métodos lo reciben por parámetro.
Opciones intercambiadas	La opción 1 llamaba a `arregloIngresado` y la 2 a `arregloDefinido`.	Se alinearon los `case` con el texto del menú.
Datos de prueba desordenados	`nums2 = {10, 6, 3, 9}` no está ordenado y el algoritmo asume orden.	Se usó `{3, 6, 9, 10}`; el resultado correcto es 5.5.

Traza de la mediana
Con `nums1 = [2,3,5,6]` y `nums2 = [3,6,9,10]`: partición X=2, Y=2 deja izquierda [2,3] y [3,6], derecha [5,6] y [9,10]. `maxLeftY=6 > minRightX=5`, así que se mueve `low` a 3. Con X=3, Y=1: izquierda [2,3,5] y [3], derecha [6] y [6,9,10]. Se cumple la condición y como hay 8 elementos, `(max(5,3) + min(6,6)) / 2 = 5.5`.

Conceptos usados

`int[]`, `Scanner`, `while`, `switch`, `Arrays.toString`, `búsqueda binaria`, `Integer.MIN_VALUE`, `Integer.MAX_VALUE`, `Math.max`, `Math.min`, `casting (double)`.

4. Listas: filtrar, aplanar, duplicados y Top N

Cuatro ejercicios de streams sobre listas. `FiltrarTransformarListas` deja los nombres que empiezan por "A", los pone en mayúsculas y los ordena. `EstructurasAnidadas` aplana una lista de listas, filtra los pares y los ordena de mayor a menor. `NumerosRepetidos` devuelve los valores que aparecen más de una vez. `TopN` devuelve los 3 productos más vendidos.

Archivos

Archivo	Qué contiene
Listas/src/listas/Listas.java	`main` que llama a los cuatro ejercicios.
Listas/src/listas/FiltrarTransformarListas.java	`filter` + `map` + `sorted` + `toList`.
Listas/src/listas/EstructurasAnidadas.java	`flatMap` + `filter` + `sorted(reverseOrder)`.
Listas/src/listas/NumerosRepetidos.java	Dos soluciones: `HashSet` y `Collections.frequency`.
Listas/src/listas/TopN.java	`groupingBy` + `summingInt` + `sorted` + `limit`.

Paso a paso

1. En FiltrarTransformar, recorrer los nombres, comparar en minúsculas con `startsWith("a")`, transformar a mayúsculas, ordenar y materializar con `toList()`.
2. En EstructurasAnidadas, `flatMap(List::stream)` convierte `List<List<Integer>>` en un solo `Stream<Integer>`.
3. En NumerosRepetidos, la primera versión usa `vistos.add(n)`: si devuelve `false`, el número ya estaba.
4. La segunda versión usa `Collections.frequency(numeros, n) > 1` y recoge en un `Set` para no repetir el resultado.
5. En TopN, agrupar por producto y sumar cantidades; luego ordenar las entradas por valor de mayor a menor y tomar 3.
6. El `main` imprime los resultados de los cuatro para comprobarlos de un vistazo.

Pseudocódigo / traza

```
FiltrarTransformar:
    nombres.stream()
        .filter(n -> n.toLowerCase().startsWith("a"))
        .map(String::toUpperCase)
        .sorted()
        .toList()

EstructurasAnidadas:
    datos.stream()
        .flatMap(List::stream)
        .filter(n -> n % 2 == 0)
        .sorted(Comparator.reverseOrder())
        .toList()

NumerosRepetidos (HashSet):
    para cada n en numeros:
        si vistos.add(n) es falso:
            duplicados.add(n)
    regresar new ArrayList<>(duplicados)

NumerosRepetidos (frequency):
    numeros.stream()
        .filter(n -> Collections.frequency(numeros, n) > 1)
        .collect(Collectors.toSet())

TopN:
    ventas.stream().collect(groupingBy(Venta::producto, summingInt(Venta::cantidad)))
    .entrySet().stream()
        .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
        .limit(3)
        .map(Map.Entry::getKey)
        .toList()
```

FiltrarTransformarListas.java

```
package listas;

import java.util.List;

public class FiltrarTransformarListas {

    public static List<String> filtrarYTransformar(){
        List<String> nombres = List.of("Ana", "pedro", "Antonio", "Luis", "alberto");

        return nombres.stream()
                      .filter(n -> n.toLowerCase().startsWith("a"))
                      .map(String::toUpperCase)
                      .sorted()
                      .toList();
    }
}
```

EstructurasAnidadas.java

```java
package listas;

import java.util.Comparator;
import java.util.List;

public class EstructurasAnidadas {

    public static List<Integer> estructAnidadas(){
        List<List<Integer>> datos = List.of(
            List.of(1, 2, 3),
            List.of(4, 5, 6),
            List.of(7, 8, 9)
        );

        return datos.stream()
                    .flatMap(List::stream)                // List<List<Integer>> ->
                        Stream<Integer>
                    .filter(n -> n % 2 == 0)              // solo pares
                    .sorted(Comparator.reverseOrder())    // de mayor a menor
                    .toList();
    }
}
```

NumerosRepetidos.java

```java
package listas;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

public class NumerosRepetidos {

    public static List<Integer> numerosRepetidos(){
        List<Integer> numeros = List.of(1, 2, 3, 2, 4, 5, 1, 6);

        Set<Integer> vistos = new HashSet<>();
        Set<Integer> duplicados = new HashSet<>();

        for(Integer n : numeros){
            // add devuelve false si el elemento ya estaba.
            if(!vistos.add(n)) duplicados.add(n);
        }

        return new ArrayList<>(duplicados);
    }

    public static Set<Integer> numerosConjuntos(){
        List<Integer> numeros = List.of(1, 2, 3, 2, 4, 5, 1, 6);

        return numeros.stream()
                      .filter(n -> Collections.frequency(numeros, n) > 1)
                      .collect(Collectors.toSet());
    }
}
```

TopN.java

```java
package listas;

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class TopN {

    public static List<String> top(){
        record Venta(String producto, int cantidad) {};

        List<Venta> ventas = List.of(
            new Venta("Leche", 50),
            new Venta("Shampoo", 34),
            new Venta("Jamon", 40),
            new Venta("Cereal", 30)
        );

        // Producto -> total de unidades vendidas.
        Map<String, Integer> totalProducto = ventas.stream()
            .collect(Collectors.groupingBy(
                Venta::producto,
                Collectors.summingInt(Venta::cantidad)
            ));

        // Ordenar las entradas por valor (de mayor a menor), tomar 3 y quedarse con la clave.
        return totalProducto.entrySet().stream()
                            .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
                            .limit(3)
                            .map(Map.Entry::getKey)
                            .toList();
    }
}
```

Listas.java (main)

```java
package listas;

import java.util.List;
import java.util.Set;

public class Listas {
    public static void main(String[] args){
        List<String> resultado = FiltrarTransformarListas.filtrarYTransformar();
        List<Integer> numRepetidos = NumerosRepetidos.numerosRepetidos();
        Set<Integer> numConjuntos = NumerosRepetidos.numerosConjuntos();
        List<Integer> anidados = EstructurasAnidadas.estructAnidadas();
        List<String> top = TopN.top();

        System.out.println("Filtrar y transformar: " + resultado);
        System.out.println("Duplicados (con for): " + numRepetidos);
        System.out.println("Duplicados (con frequency + Set): " + numConjuntos);
        System.out.println("Estructuras anidadas: " + anidados);
        System.out.println("Top 3 productos: " + top);
    }
}
```

Salida esperada

```
Filtrar y transformar: [ALBERTO, ANA, ANTONIO]
Duplicados (con for): [1, 2]
Duplicados (con frequency + Set): [1, 2]
Estructuras anidadas: [8, 6, 4, 2]
Top 3 productos: [Leche, Jamon, Shampoo]
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
main incompleto	Solo se llamaban dos de los cuatro ejercicios.	Se añadieron `EstructurasAnidadas.estructAnidadas()` y `TopN.top()`.
Resultado de duplicados sin orden	`HashSet` no garantiza orden; el resultado podría variar.	Para este ejemplo sale `[1, 2]`, pero si necesitas orden usa `TreeSet` o `sorted()`.

Traza de EstructurasAnidadas
`flatMap` produce 1,2,3,4,5,6,7,8,9. `filter` deja 2,4,6,8. `sorted(reverseOrder)` produce 8,6,4,2. `toList()` lo materializa en una lista inmutable.

Empates en TopN
Si dos productos tienen la misma cantidad, el orden entre ellos no está garantizado por el comparador. Para un desempate estable añade un segundo criterio, por ejemplo `.thenComparing(Map.Entry.comparingByKey())`.

Conceptos usados

`List`, `Set`, `HashSet`, `ArrayList`, `stream`, `filter`, `map`, `flatMap`, `sorted`, `Comparator.reverseOrder`, `toList`, `collect`, `Collectors.toSet`, `Collections.frequency`, `groupingBy`, `summingInt`, `Map.Entry.comparingByValue`, `limit`, `record local`.

5. Contar la frecuencia de palabras

Dada una frase, devolver un `Map<String, Integer>` con cuántas veces aparece cada palabra. Ejemplo: `"spring boot es genial spring es rapido"` -> `{spring=2, boot=1, es=2, genial=1, rapido=1}`.

Archivos

Archivo	Qué contiene
Mapas/src/mapas/RepetirPalabras.java	`split`, `Arrays.stream` y `Collectors.toMap`.
Mapas/src/mapas/Mapas.java	`main` que imprime el mapa.

Paso a paso

1. Partir la frase en palabras con `frase.split("\\s+")`.
2. Convertir el arreglo en stream con `Arrays.stream(...)`.
3. Usar `Collectors.toMap` con tres argumentos: clave = la palabra, valor = 1, y función de merge = `Integer::sum`.
4. Cuando una palabra se repite, el merge suma 1 al valor que ya estaba.
5. Imprimir el mapa resultante.

Pseudocódigo / traza

```
palabras = frase.split("\s+")
mapa = {}
para cada palabra en palabras:
    si mapa contiene palabra:
        mapa[palabra] = mapa[palabra] + 1
    si no:
        mapa[palabra] = 1
regresar mapa

Traza:
"spring" -> {spring=1}
"boot"   -> {spring=1, boot=1}
"es"     -> {spring=1, boot=1, es=1}
"genial" -> {spring=1, boot=1, es=1, genial=1}
"spring" -> {spring=2, boot=1, es=1, genial=1}
"es"     -> {spring=2, boot=1, es=2, genial=1}
"rapido" -> {spring=2, boot=1, es=2, genial=1, rapido=1}
```

RepetirPalabras.java

```
package mapas;

import java.util.Arrays;
import java.util.Map;
import java.util.stream.Collectors;

public class RepetirPalabras {

    public static Map<String, Integer> frecuencia(){
        String frase = "spring boot es genial spring es rapido";

        return Arrays.stream(frase.split("\\s+"))
                     .collect(Collectors.toMap(
                         w -> w,        // clave: la palabra
                         w -> 1,        // valor inicial: 1
                         Integer::sum   // si la clave se repite, suma los valores
                     ));
    }
}
```

Mapas.java

```
1  | package mapas;
2  |
3  | import java.util.Map;
4  |
5  | public class Mapas {
6  |     public static void main(String[] args){
7  |         Map<String, Integer> frecuencia = RepetirPalabras.frecuencia();
8  |         System.out.println("El resultado es: " + frecuencia);
9  |     }
10 | }
```

Salida esperada

```
El resultado es: {rapido=1, spring=2, genial=1, boot=1, es=2}
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
La frase no coincidía con el resultado esperado	El comentario esperaba `spring=2`, pero la cadena solo tenía `spring` una vez.	Se agregó la segunda aparición de `spring` en la frase.
Orden del mapa	`HashMap` no garantiza el orden de impresión.	El contenido es correcto; si necesitas orden, usa `TreeMap` o `LinkedHashMap`.

Alternativa con groupingBy
También se puede escribir `Collectors.groupingBy(w -> w, Collectors.counting())`, que devuelve `Map<String, Long>`. `toMap` es más directo cuando ya sabes que el valor inicial es 1.

Conceptos usados

`String.split`, `Arrays.stream`, `Collectors.toMap`, `Integer::sum`, `HashMap`, `Map`.

6. Agrupar personas por ciudad y contarlas

Dada una lista de `Persona(nombre, edad, ciudad)`, agruparlas por ciudad y contar cuántas hay en cada una. El resultado es `Map<String, Long>`.

Archivos

Archivo	Qué contiene
Colectores/src/colectores/AgrupacionColectores.java	`groupingBy` + `counting`.
Colectores/src/colectores/Colectores.java	`main` que imprime el mapa.

Paso a paso

1. Definir el record local `Persona` con sus tres campos.
2. Crear una lista de personas de ejemplo.
3. Hacer `personas.stream().collect(groupingBy(Persona::ciudad, counting()))`.
4. El clasificador `Persona::ciudad` decide la clave; `counting()` cuenta los elementos de cada grupo.
5. Imprimir el `Map<String, Long>`.

Pseudocódigo / traza

```
mapa = {}
para cada persona en personas:
    ciudad = persona.ciudad()
    mapa[ciudad] = mapa.getOrDefault(ciudad, 0) + 1
regresar mapa

Traza:
Ana  -> Madrid=1
Luis -> Madrid=2
Eva  -> Barcelona=1
Resultado: {Madrid=2, Barcelona=1}
```

AgrupacionColectores.java

```java
package colectores;

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class AgrupacionColectores {

    public static Map<String, Long> personaCiudad(){
        record Persona(String nombre, int edad, String ciudad) {};

        List<Persona> personas = List.of(
            new Persona("Ana", 25, "Madrid"),
            new Persona("Luis", 30, "Madrid"),
            new Persona("Eva", 22, "Barcelona")
        );

        return personas.stream()
                       .collect(Collectors.groupingBy(
                           Persona::ciudad,       // clave del mapa
                           Collectors.counting()  // valor: cuántos hay
                       ));
    }
}
```

Colectores.java

```
package colectores;

import java.util.Map;

public class Colectores {
    public static void main(String[] args){
        Map<String, Long> resultado = AgrupacionColectores.personaCiudad();
        System.out.println("El resultado del registro es: " + resultado);
    }
}
```

Salida esperada

```
El resultado del registro es: {Barcelona=1, Madrid=2}
```

El downstream collector

El segundo argumento de `groupingBy` es lo que se hace con cada grupo. `counting()` cuenta; `summingInt(...)` suma; `mapping(...)` transforma antes de contar; `toList()` (por defecto) guarda las personas.

Conceptos usados

`record local`, `List.of`, `stream`, `collect`, `Collectors.groupingBy`, `Collectors.counting`, `Map`.

7. Precio promedio con filtros

Dada una lista de `Producto(nombre, precio, categoria)`, calcular el precio promedio de los productos de categoría `Electronica` cuyo precio sea mayor que 100.

Archivos

Archivo	Qué contiene
Promedio/src/promedio/SumaPromedio.java	`filter` + `mapToDouble` + `average`.
Promedio/src/promedio/Inicial.java	`main` que imprime el promedio.

Paso a paso

1. Filtrar por categoría con `"Electronica".equalsIgnoreCase(p.categoria())`.
2. Filtrar por precio con `p.precio() > 100`.
3. Transformar a `double` con `mapToDouble(Producto::precio)`.
4. Calcular el promedio con `average()`, que devuelve `OptionalDouble`.
5. Usar `orElse(0.0)` para el caso de que no quede ningún producto.

Pseudocódigo / traza

```
productos = [Celular 4500 Electronica, TV 10000 Electronica, Arroz 300 Alimentos]

1. filter categoria == "Electronica" -> Celular, TV
2. filter precio > 100               -> Celular, TV
3. mapToDouble(precio)               -> 4500.0, 10000.0
4. average()                         -> (4500 + 10000) / 2 = 7250.0
5. orElse(0.0)                       -> 7250.0
```

SumaPromedio.java

```java
package promedio;

import java.util.List;

public class SumaPromedio {

    public static double sumPro(){
        record Producto(String nombre, double precio, String categoria) {};

        List<Producto> productos = List.of(
            new Producto("Celular Xiomi", 4500, "Electronica"),
            new Producto("Smart TV Samsung", 10000, "Electronica"),
            new Producto("Arroz", 300, "Alimentos")
        );

        return productos.stream()
                        .filter(p -> "Electronica".equalsIgnoreCase(p.categoria()))
                        .filter(p -> p.precio() > 100)
                        .mapToDouble(Producto::precio)
                        .average()
                        .orElse(0.0);
    }
}
```

Inicial.java

```
package promedio;

public class Inicial {
    public static void main(String[] args){
        double resultado = SumaPromedio.sumPro();
        System.out.println("El promedio es: " + resultado);
    }
}
```

Salida esperada

```
El promedio es: 7250.0
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
Filtro de categoría equivocado	El código filtraba `"Alimentos"`, así que solo entraba el Arroz (300) y el promedio era 300.0.	Se cambió a `"Electronica".equalsIgnoreCase(...)`; ahora entran Celular y TV.
Posible división por cero	Si ningún producto cumple los filtros, `average()` devuelve un `OptionalDouble` vacío.	`orElse(0.0)` evita el error y devuelve 0.0.

¿Por qué dos filter?
Se podrían combinar en uno con `&&`. Separarlos hace más legible cada condición y permite ver en la traza qué productos pasan cada etapa.

Conceptos usados

`record local`, `List.of`, `stream`, `filter`, `mapToDouble`, `average`, `OptionalDouble`, `orElse`, `equalsIgnoreCase`.

8. Patrón Strategy: descuentos intercambiables

Crear una interfaz `Descuento` con tres implementaciones: `SinDescuento` (0%), `DescuentoNavidad` (20%) y `DescuentoBlackFriday` (50%). Un servicio recibe la implementación por constructor y calcula el precio final. Es el mismo esquema que usarás con `@Service` e inyección de dependencias en Spring.

Archivos

Archivo	Qué contiene
Descuento/src/descuento/Interfaces/Descuento.java	Contrato: `double aplicar(double precio)`.
Descuento/src/descuento/Implementaciones/*.java	Tres estrategias concretas.
Descuento/src/descuento/Service/DescuentoService.java	Recibe un `Descuento` y delega el cálculo.
Descuento/src/descuento/DescuentoMain.java	Crea las tres combinaciones y compara.

Paso a paso

1. Definir la interfaz `Descuento` con el método `aplicar`.
2. Implementar cada estrategia con su propia fórmula: `precio`, `precio * 0.80`, `precio * 0.50`.
3. Crear `DescuentoService` con un campo `private final Descuento descuento`.
4. En el constructor, recibir la implementación y guardarla.
5. `calcular(precio)` simplemente llama a `descuento.aplicar(precio)`: polimorfismo.
6. En el `main`, crear tres servicios, cada uno con una estrategia distinta, y comparar resultados.

Pseudocódigo / traza

```
interfaz Descuento:
    double aplicar(double precio)

clase DescuentoNavidad implementa Descuento:
    aplicar(precio) -> precio * 0.80

clase DescuentoBlackFriday implementa Descuento:
    aplicar(precio) -> precio * 0.50

clase SinDescuento implementa Descuento:
    aplicar(precio) -> precio

servicio DescuentoService:
    constructor(Descuento d) { this.descuento = d }
    calcular(precio) { return descuento.aplicar(precio) }

main:
    sinDes  = new DescuentoService(new SinDescuento())
    navidad = new DescuentoService(new DescuentoNavidad())
    black   = new DescuentoService(new DescuentoBlackFriday())

    sinDes.calcular(100)  -> 100.0
    navidad.calcular(100) -> 80.0
    black.calcular(100)   -> 50.0
```

Descuento.java

```java
package descuento.Interfaces;

public interface Descuento {
    double aplicar(double precio);
}
```

Implementaciones

```
package descuento.Implementaciones;

import descuento.Interfaces.Descuento;

public class SinDescuento implements Descuento {
    public double aplicar(double precio){
        return precio;
    }
}

// ------------------------------------------------------------

package descuento.Implementaciones;

import descuento.Interfaces.Descuento;

public class DescuentoNavidad implements Descuento {
    public double aplicar(double precio){
        return precio * 0.80;   // 20% de descuento
    }
}

// ------------------------------------------------------------

package descuento.Implementaciones;

import descuento.Interfaces.Descuento;

public class DescuentoBlackFriday implements Descuento {
    public double aplicar(double precio){
        return precio * 0.50;   // 50% de descuento
    }
}
```

DescuentoService.java

```
package descuento.Service;

import descuento.Interfaces.Descuento;

public class DescuentoService {
    private final Descuento descuento;

    // Inyección por constructor: el servicio no decide qué descuento usar.
    public DescuentoService(Descuento descuento){
        this.descuento = descuento;
    }

    public double calcular(double precio){
        return descuento.aplicar(precio); // despacho dinámico
    }
}
```

DescuentoMain.java

```java
package descuento;

import descuento.Implementaciones.DescuentoBlackFriday;
import descuento.Implementaciones.DescuentoNavidad;
import descuento.Implementaciones.SinDescuento;
import descuento.Service.DescuentoService;

public class DescuentoMain {
    public static void main(String[] args){
        DescuentoService sinDes     = new DescuentoService(new SinDescuento());
        DescuentoService desNavidad = new DescuentoService(new DescuentoNavidad());
        DescuentoService desBlack   = new DescuentoService(new DescuentoBlackFriday());

        System.out.println("Sin descuento: " + sinDes.calcular(100));
        System.out.println("Black Friday: " + desBlack.calcular(100));
        System.out.println("Navidad: " + desNavidad.calcular(100));
    }
}
```

Salida esperada

```
El precio del producto sin descuento es: 100.0.
El precio del producto con descuento de blackfriday es: 50.0.
El precio del producto con descuento de navidad es: 80.0.
```

¿Por qué Strategy?
Si mañana agregas `DescuentoEstudiante`, solo creas otra clase que implemente `Descuento`; no modificas `DescuentoService` ni el `main`. Eso es el principio abierto/cerrado. En Spring, el contenedor inyecta la implementación y tú solo pides la interfaz.

Conceptos usados

`interface`, `implements`, `polimorfismo`, `inyección por constructor`, `private final`, `Strategy`.

9. Inyección de dependencias sin Spring

Simular la inyección de dependencias: `NotificadorService` no crea su propio enviador, sino que recibe una implementación de `EmailSenderInterface` por constructor. El `main` decide si usa `SmtpSender` (o cualquier otra implementación).

Archivos

Archivo	Qué contiene
InyeccionDeDependencias/src/inyeccionDeDependencias/Interfaces/EmailSenderInterface.java	Contrato `enviar(destino, mensaje)`.
.../Implementaciones/SmtpSender.java	Implementación concreta que imprime por consola.
.../Service/NotificadorService.java	Depende de la interfaz, no de la implementación.
.../InyeccionInicial.java	`main` que inyecta `SmtpSender`.

Paso a paso

1. Definir la interfaz `EmailSenderInterface` con el método `enviar`.
2. Crear `SmtpSender`, que implementa la interfaz.
3. Crear `NotificadorService` con un campo `EmailSenderInterface` y constructor que lo recibe.
4. En `notificar`, delegar en `emailSender.enviar(...)`.
5. En el `main`, crear el sender, pasarlo al servicio y llamar a `notificar`.

Pseudocódigo / traza

```
interfaz EmailSenderInterface:
    void enviar(String destino, String mensaje)

clase SmtpSender implementa EmailSenderInterface:
    enviar(destino, mensaje):
        imprimir "SMTP -> destino: mensaje"

clase NotificadorService:
    campo emailSender: EmailSenderInterface
    constructor(EmailSenderInterface e) { emailSender = e }
    notificar(email, msge) { emailSender.enviar(email, msge) }

main:
    sender = new SmtpSender()
    service = new NotificadorService(sender)
    service.notificar("erik@gmail.com", "Hola")
```

Interfaz e implementación

```
package inyeccionDeDependencias.Interfaces;

public interface EmailSenderInterface {
    void enviar(String destino, String mensaje);
}

// ------------------------------------------------------------

package inyeccionDeDependencias.Implementaciones;

import inyeccionDeDependencias.Interfaces.EmailSenderInterface;

public class SmtpSender implements EmailSenderInterface {
    public void enviar(String destino, String mensaje){
        System.out.println("SMTP -> " + destino + ": " + mensaje);
    }
}
```

NotificadorService.java

```java
package inyeccionDeDependencias.Service;

import inyeccionDeDependencias.Interfaces.EmailSenderInterface;

public class NotificadorService {
    private final EmailSenderInterface emailSender;

    // La dependencia entra desde fuera: esto es inyección por constructor.
    public NotificadorService(EmailSenderInterface emailSender){
        this.emailSender = emailSender;
    }

    public void notificar(String email, String msge){
        emailSender.enviar(email, msge);
    }
}
```

InyeccionInicial.java

```java
package inyeccionDeDependencias;

import inyeccionDeDependencias.Implementaciones.SmtpSender;
import inyeccionDeDependencias.Interfaces.EmailSenderInterface;
import inyeccionDeDependencias.Service.NotificadorService;

public class InyeccionInicial {
    public static void main(String[] args){
        EmailSenderInterface sender = new SmtpSender();
        NotificadorService service = new NotificadorService(sender);
        service.notificar("erik@gmail.com", "Hola");
    }
}
```

Salida esperada

```
SMTP -> erik@gmail.com: Hola
```

La inversión de control
`NotificadorService` no sabe si el correo sale por SMTP, por una API o por un mock de pruebas. Solo sabe que alguien cumple el contrato `EmailSenderInterface`. Esa es la base de la testabilidad y de la inyección de dependencias de Spring.

Conceptos usados

`interface`, `implements`, `inyección de dependencias`, `constructor`, `private final`, `polimorfismo`.

10. Mini-repositorio en memoria

Crear `UsuarioRepository` con un `Map<Long, Usuario>` que soporte `save` (asigna ID automático), `findById` (devuelve `Optional<Usuario>`), `findAll` y `deleteById`. Es el comportamiento básico que Spring Data JPA genera por debajo.

Archivos

Archivo	Qué contiene
Repositorio/src/repositorio/UsuarioRepository.java	`Map`, `AtomicLong` y `Optional`.
Repositorio/src/repositorio/RepositorioInicial.java	`main` con el ciclo completo: guardar, buscar, listar, actualizar y borrar.

Paso a paso

1. Crear el mapa `db` y el contador `AtomicLong` que empieza en 1.
2. En `save`, si el usuario viene con `id == null`, crear uno nuevo con el ID generado.
3. Guardar siempre en el mapa con `db.put(u.id(), u)`; así también se actualiza.
4. En `findById`, usar `Optional.ofNullable(db.get(id))` para no devolver `null`.
5. En `findAll`, devolver `new ArrayList<>(db.values())` para no exponer el mapa interno.
6. En `deleteById`, usar `db.remove(id)`.
7. El `main` demuestra el ciclo completo y comprueba el ID inexistente.

Pseudocódigo / traza

```
save(u):
    si u.id == null:
        u = new Usuario(contador.getAndIncrement(), u.nombre, u.email)
    db[u.id] = u
    regresar u

findById(id):
    regresar Optional.ofNullable(db[id])

findAll():
    regresar new ArrayList<>(db.values())

deleteById(id):
    db.remove(id)

Traza:
save(Ana sin id)  -> id=1
save(Luis sin id) -> id=2
save(Eva sin id)  -> id=3
findById(2)       -> Optional[Luis]
save(Usuario(1, "Ana María", ...)) -> reemplaza el id 1
deleteById(2)     -> quedan 1 y 3
```

UsuarioRepository.java

```
package repositorio;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.atomic.AtomicLong;

// record de paquete: se usa desde clases del mismo paquete.
record Usuario(Long id, String nombre, String email) {};

public class UsuarioRepository {
    private final Map<Long, Usuario> db = new HashMap<>();

    // AtomicLong evita condiciones de carrera al generar IDs.
    private final AtomicLong contador = new AtomicLong(1);

    public Usuario save(Usuario u){
        if(u.id() == null){
            u = new Usuario(contador.getAndIncrement(), u.nombre(), u.email());
        }

        db.put(u.id(), u); // inserta o actualiza
        return u;
    }

    public Optional<Usuario> findById(Long id){
        return Optional.ofNullable(db.get(id));
    }

    public List<Usuario> findAll(){
        return new ArrayList<>(db.values()); // copia defensiva
    }

    public void deleteById(Long id) {
        db.remove(id);
    }
}
```

RepositorioInicial.java (main)

```
package repositorio;

public class RepositorioInicial {
    public static void main(String[] args){
        UsuarioRepository repo = new UsuarioRepository();

        Usuario ana  = repo.save(new Usuario(null, "Ana", "ana@mail.com"));
        Usuario luis = repo.save(new Usuario(null, "Luis", "luis@mail.com"));
        repo.save(new Usuario(null, "Eva", "eva@mail.com"));

        System.out.println("Guardados: " + ana + " y " + luis);

        repo.findById(2L).ifPresent(u -> System.out.println("Encontrado: " + u));
        System.out.println("¿Existe el ID 99? " + repo.findById(99L).isPresent());

        System.out.println("Todos: " + repo.findAll());

        // Actualización: mismo ID, nuevos datos.
        repo.save(new Usuario(1L, "Ana María", "ana.maria@mail.com"));
        System.out.println("Después de actualizar: " + repo.findById(1L).orElseThrow());

        repo.deleteById(2L);
        System.out.println("Después de borrar el 2: " + repo.findAll());
    }
}
```

Salida esperada

```
Guardados: Usuario[id=1, nombre=Ana, email=ana@mail.com] y Usuario[id=2, nombre=Luis,
    email=luis@mail.com]
Encontrado: Usuario[id=2, nombre=Luis, email=luis@mail.com]
¿Existe el ID 99? false
Todos: [Usuario[id=1, ...], Usuario[id=2, ...], Usuario[id=3, ...]]
Después de actualizar: Usuario[id=1, nombre=Ana María, email=ana.maria@mail.com]
Después de borrar el 2: [Usuario[id=1, ...], Usuario[id=3, ...]]
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
`RepositorioInicial` vacío	No había `main` para probar el repositorio.	Se añadió un `main` con el ciclo guardar, buscar, listar, actualizar y borrar.
`HashMap` no es thread-safe	El contador es atómico, pero el mapa no. En un entorno con varios hilos podría corromperse.	Para este ejercicio es suficiente; en producción se usaría `ConcurrentHashMap` o una base de datos.

¿Qué hace Spring Data JPA?
Spring genera una implementación de la interfaz del repositorio y traduce métodos como `findById`, `findAll`, `save` y `deleteById` a SQL. Este ejercicio reproduce esa idea con un mapa en memoria.

Conceptos usados

`record`, `Map`, `HashMap`, `AtomicLong`, `Optional`, `ofNullable`, `ifPresent`, `orElseThrow`, `ArrayList`, `copia defensiva`.

11. API de tareas en consola por capas

Aplicación de consola para crear, listar, completar y borrar tareas. Usa `Map<Long, Tarea>` como base de datos y separa en capas: `ApiModel` (datos), `ApiRepository` (persistencia), `ApiService` (reglas) y `ApiController` (interacción con el usuario).

Archivos

Archivo	Qué contiene
ApiConsola/src/apiConsola/Model/ApiModel.java	Record `Tarea`.
.../Repository/ApiRepository.java	`Map`, `AtomicLong`, `Optional`.
.../Service/ApiService.java	Validación y reglas de negocio.
.../Controller/ApiController.java	Menú de consola y `Scanner`.
.../Inicial.java	`main` que arranca el controlador.

Paso a paso

1. El modelo define `Tarea(id, titulo, completada)`.
2. El repositorio guarda las tareas en un mapa y genera IDs con `AtomicLong`.
3. El servicio valida el título, crea la tarea, la completa o la borra; no sabe nada de consola.
4. El controlador muestra el menú, lee la opción y llama al servicio.
5. El `main` solo crea el controlador y llama a `iniciar()`.
6. Flujo crear: leer título -> `service.crear(titulo)` -> `repo.save(...)` -> ID generado.
7. Flujo completar: buscar por ID -> si no existe, `orElseThrow` -> guardar copia con `completada = true`.

Pseudocódigo / traza

```
crear(titulo):
    si titulo vacío: lanzar IllegalArgumentException
    regresar repo.save(new Tarea(null, titulo, false))

save(t):
    si t.id == null:
        t = new Tarea(seq.getAndIncrement(), t.titulo, t.completada)
    db[t.id] = t
    regresar t

completar(id):
    t = repo.findById(id).orElseThrow(() -> RuntimeException("No existe"))
    regresar repo.save(new Tarea(t.id, t.titulo, true))

listar(): regresar repo.findAll()
borrar(id): repo.delete(id)

Traza:
crear("Comprar leche") -> Tarea[1, Comprar leche, false]
listar()               -> [Tarea[1, ...]]
completar(1)           -> Tarea[1, Comprar leche, true]
borrar(1)              -> lista vacía
```

ApiModel.java

```java
package apiConsola.Model;

public class ApiModel {
    // Record anidado y público para que otras capas puedan usarlo.
    public record Tarea(Long id, String titulo, boolean completada) {
    }
}
```

ApiRepository.java

```java
package apiConsola.Repository;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.atomic.AtomicLong;

import apiConsola.Model.ApiModel.Tarea;

public class ApiRepository {
    private final Map<Long, Tarea> db = new HashMap<>();
    private final AtomicLong seq = new AtomicLong(1);

    public Tarea save(Tarea t){
        if(t.id() == null){
            t = new Tarea(seq.getAndIncrement(), t.titulo(), t.completada());
        }
        db.put(t.id(), t); // siempre guarda: sirve para crear y actualizar
        return t;
    }

    public Optional<Tarea> findById(Long id){
        return Optional.ofNullable(db.get(id));
    }

    public List<Tarea> findAll(){
        return new ArrayList<>(db.values());
    }

    public void delete(Long id){
        db.remove(id);
    }
}
```

ApiService.java

```java
package apiConsola.Service;

import java.util.List;

import apiConsola.Model.ApiModel.Tarea;
import apiConsola.Repository.ApiRepository;

public class ApiService {
    private final ApiRepository repo;

    public ApiService(ApiRepository repo){
        this.repo = repo;
    }

    public Tarea crear(String titulo){
        if(titulo == null || titulo.isBlank()){
            throw new IllegalArgumentException("Titulo vacío");
        }
        return repo.save(new Tarea(null, titulo, false));
    }

    public Tarea completar(Long id){
        Tarea t = repo.findById(id)
                      .orElseThrow(() -> new RuntimeException("No existe la tarea " + id));

        return repo.save(new Tarea(t.id(), t.titulo(), true));
    }

    public List<Tarea> listar(){
        return repo.findAll();
    }

    public void borrar(Long id){
        repo.delete(id);
    }
}
```

ApiController.java

```
package apiConsola.Controller;

import java.util.Scanner;

import apiConsola.Repository.ApiRepository;
import apiConsola.Service.ApiService;

public class ApiController {
    private final ApiService service = new ApiService(new ApiRepository());
    private final Scanner sc = new Scanner(System.in);

    // El bucle va dentro de un método, no suelto en el cuerpo de la clase.
    public void iniciar(){
        boolean salir = false;

        while(!salir){
            System.out.println("\n1.Crear 2.Listar 3.Completar 4.Borrar 5.Salir");
            System.out.print("Opción: ");
            String opcion = sc.nextLine().trim();

            switch(opcion){
                case "1" -> {
                    System.out.print("Titulo: ");
                    String titulo = sc.nextLine();
                    System.out.println("Creada: " + service.crear(titulo));
                }
                case "2" -> service.listar().forEach(System.out::println);
                case "3" -> {
                    System.out.print("ID: ");
                    Long id = Long.parseLong(sc.nextLine().trim());
                    System.out.println("Completada: " + service.completar(id));
                }
                case "4" -> {
                    System.out.print("ID: ");
                    Long id = Long.parseLong(sc.nextLine().trim());
                    service.borrar(id);
                    System.out.println("Borrada la tarea " + id);
                }
                case "5" -> salir = true;
                default -> System.out.println("Opción no válida.");
            }
        }
    }
}
```

Inicial.java

```
package apiConsola;

import apiConsola.Controller.ApiController;

public class Inicial {
    public static void main(String[] args){
        new ApiController().iniciar();
    }
}
```

Salida esperada

```
1.Crear 2.Listar 3.Completar 4.Borrar 5.Salir
Opción: 1
Titulo: Comprar leche
Creada: Tarea[id=1, titulo=Comprar leche, completada=false]

1.Crear 2.Listar 3.Completar 4.Borrar 5.Salir
Opción: 2
Tarea[id=1, titulo=Comprar leche, completada=false]

1.Crear 2.Listar 3.Completar 4.Borrar 5.Salir
Opción: 3
ID: 1
Completada: Tarea[id=1, titulo=Comprar leche, completada=true]
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
`while` en el cuerpo de la clase	En Java no puede haber sentencias sueltas fuera de un método; el archivo no compilaba.	Se envolvió el bucle en `public void iniciar()` y el `main` lo llama.
`orElseThrow -> ...`	Faltaban los paréntesis de la lambda; la sintaxis era inválida.	`orElseThrow(() -> new RuntimeException("No existe"))`.
Faltaba importar `Tarea`	`Tarea` es un record anidado en `ApiModel`, no está en el mismo paquete.	`import apiConsola.Model.ApiModel.Tarea;` en Repository y Service.
`save` no actualizaba	Solo hacía `db.put` cuando el ID era `null`; al completar una tarea existente no se guardaba.	Se mueve `db.put(t.id(), t)` fuera del `if`.
`next()` cortaba el título	`Scanner.next()` lee solo una palabra; "Comprar leche" se guardaba como "Comprar".	Se usa `nextLine()` y se lee la opción como texto para evitar el salto pendiente.
`Inicial` vacío	No había punto de entrada.	`main` que crea `ApiController` y llama a `iniciar()`.

Por qué separar en capas
El controlador no sabe cómo se guarda; el servicio no sabe cómo se lee; el repositorio no sabe de menús. Cada capa se puede probar o cambiar por separado. En Spring, esas capas se anotan con `@RestController`, `@Service` y `@Repository`.

Conceptos usados

`capas`, `record anidado`, `Map`, `HashMap`, `AtomicLong`, `Optional`, `orElseThrow`, `Scanner`, `nextLine`, `switch con flechas`, `lambda`, `forEach`, `method reference`.

12. Sistema de pedidos con relaciones

Modelar `Cliente`, `Producto`, `LineaPedido` y `Pedido`. Calcular el total de un pedido, el cliente que más gastó y el producto más vendido por unidades. Es el ejercicio que más usa `groupingBy`, `flatMap` y referencias a métodos como `Pedido::cliente`.

Archivos

Archivo	Qué contiene
Pedidos/src/pedidos/Repository/Cliente.java	`record Cliente(Long id, String nombre)`.
.../Producto.java	`record Producto(Long id, String nombre, double precio)`.
.../LineaPedido.java	Producto + cantidad; `subtotal()`.
.../Pedido.java	Cliente + líneas; `total()`.
.../PedidosRepository.java	Agrupaciones y cálculos.
Pedidos/src/pedidos/Inicial.java	Datos de ejemplo y salida.

Paso a paso

1. Modelar cada entidad como record; `LineaPedido` conoce su subtotal y `Pedido` suma los subtotales.
2. Crear un repositorio que reciba la lista de pedidos por constructor.
3. Para el gasto por cliente: `groupingBy(Pedido::cliente, summingDouble(Pedido::total))`.
4. Para el cliente top: recorrer las entradas, `max(comparingByValue())`, quedarse con la clave y extraer el nombre.
5. Para el producto top: `flatMap` de las líneas, agrupar por nombre de producto y sumar cantidades.
6. Para el ingreso por producto: igual, pero sumando `subtotal()` en vez de cantidad.

Pseudocódigo / traza

```
total(pedido):
    suma de linea.subtotal() para cada linea

subtotal(linea):
    producto.precio * cantidad

gastoPorCliente:
    groupingBy(Pedido::cliente, summingDouble(Pedido::total))

clienteQueMasGasto:
    gastoPorCliente.entrySet()
        .max(comparingByValue())
        .map(entry -> entry.getKey().nombre())
        .orElse(null)

productoMasVendido:
    pedidos.stream()
        .flatMap(p -> p.lineas().stream())
        .collect(groupingBy(l -> l.producto().nombre(), summingInt(LineaPedido::cantidad)))
        .entrySet().stream()
        .max(comparingByValue())
        .map(Map.Entry::getKey)
        .orElse(null)

Traza:
p1: 4*2.50 + 2*8.00 = 10 + 16 = 26
p2: 3*4.00 + 2*2.50 = 12 + 5  = 17
p3: 1*8.00                    = 8
Gasto: Ana = 26 + 8 = 34; Luis = 17
Unidades: Leche = 4+2 = 6; Jamón = 2+1 = 3; Cereal = 3
```

Records del modelo

```
package pedidos.Repository;

public record Cliente(Long id, String nombre) {
}

// ------------------------------------------------------------

package pedidos.Repository;

public record Producto(Long id, String nombre, double precio) {
}

// ------------------------------------------------------------

package pedidos.Repository;

public record LineaPedido(Producto producto, int cantidad) {
    public double subtotal(){
        return producto.precio() * cantidad;
    }
}

// ------------------------------------------------------------

package pedidos.Repository;

import java.util.List;

public record Pedido(Long id, Cliente cliente, List<LineaPedido> lineas) {
    public double total(){
        return lineas.stream()
                    .mapToDouble(LineaPedido::subtotal)
                    .sum();
    }
}
```

PedidosRepository.java

```java
package pedidos.Repository;

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class PedidosRepository {
    private final List<Pedido> pedidos;

    public PedidosRepository(List<Pedido> pedidos){
        this.pedidos = pedidos;
    }

    public Map<Cliente, Double> gastoPorCliente(){
        return pedidos.stream()
                      .collect(Collectors.groupingBy(
                          Pedido::cliente,
                          Collectors.summingDouble(Pedido::total)
                      ));
    }

    public String clienteQueMasGasto(){
        return gastoPorCliente().entrySet().stream()
                                .max(Map.Entry.comparingByValue())
                                .map(entry -> entry.getKey().nombre())
                                .orElse(null);
    }

    public String productoMasVendido(){
        return pedidos.stream()
                      .flatMap(p -> p.lineas().stream())
                      .collect(Collectors.groupingBy(
                          linea -> linea.producto().nombre(),
                          Collectors.summingInt(LineaPedido::cantidad)
                      ))
                      .entrySet().stream()
                      .max(Map.Entry.comparingByValue())
                      .map(Map.Entry::getKey)
```

```java
                      .orElse(null);
    }

    public Map<String, Double> ingresoPorProducto(){
        return pedidos.stream()
                      .flatMap(p -> p.lineas().stream())
                      .collect(Collectors.groupingBy(
                          linea -> linea.producto().nombre(),
                          Collectors.summingDouble(LineaPedido::subtotal)
                      ));
    }
}
```

Inicial.java

```java
package pedidos;

import java.util.List;

import pedidos.Repository.Cliente;
import pedidos.Repository.LineaPedido;
import pedidos.Repository.Pedido;
import pedidos.Repository.PedidosRepository;
import pedidos.Repository.Producto;

public class Inicial {
    public static void main(String[] args){
        Producto leche  = new Producto(1L, "Leche", 2.50);
        Producto jamon  = new Producto(2L, "Jamón", 8.00);
        Producto cereal = new Producto(3L, "Cereal", 4.00);

        Cliente ana  = new Cliente(1L, "Ana");
        Cliente luis = new Cliente(2L, "Luis");

        Pedido p1 = new Pedido(1L, ana, List.of(
            new LineaPedido(leche, 4),
            new LineaPedido(jamon, 2)
        ));
        Pedido p2 = new Pedido(2L, luis, List.of(
            new LineaPedido(cereal, 3),
            new LineaPedido(leche, 2)
        ));
        Pedido p3 = new Pedido(3L, ana, List.of(
            new LineaPedido(jamon, 1)
        ));

        PedidosRepository repo = new PedidosRepository(List.of(p1, p2, p3));

        System.out.println("Total pedido 1: " + p1.total());
        System.out.println("Total pedido 2: " + p2.total());
        System.out.println("Total pedido 3: " + p3.total());
        System.out.println("Gasto por cliente: " + repo.gastoPorCliente());
        System.out.println("Cliente que más gastó: " + repo.clienteQueMasGasto());
        System.out.println("Producto más vendido: " + repo.productoMasVendido());
        System.out.println("Ingreso por producto: " + repo.ingresoPorProducto());
    }
}
```

Salida esperada

```
Total pedido 1: 26.0
Total pedido 2: 17.0
Total pedido 3: 8.0
Gasto por cliente: {Cliente[id=1, nombre=Ana]=34.0, Cliente[id=2, nombre=Luis]=17.0}
Cliente que más gastó: Ana
Producto más vendido: Leche
Ingreso por producto: {Jamón=24.0, Cereal=12.0, Leche=15.0}
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
Variable `pedidos` inexistente	El repositorio usaba `pedidos.stream()` sin haber declarado ni recibido la lista.	Se añadió el campo `private final List<Pedido> pedidos` y el constructor.
Records inaccesibles desde `pedidos`	Los records estaban en el paquete `pedidos.Repository` con visibilidad de paquete, y `Inicial` está en `pedidos`.	Se separaron en archivos públicos (`Cliente.java`, `Producto.java`, etc.).
`Inicial` vacío	No había datos ni `main`.	Se creó el `main` con productos, clientes y tres pedidos.

Error	Por qué fallaba	Corrección
`groupingBy` por nombre vs por objeto	Agrupar por `Cliente` funciona porque los records implementan `equals` y `hashCode`; agrupar por nombre también es válido.	Se mantuvo `Pedido::cliente` para practicar la referencia a método y se extrae el nombre al final.

Por qué `flatMap`
Cada pedido tiene una lista de líneas. Si usáramos `map`, obtendríamos un `Stream<List<LineaPedido>>`. `flatMap(p -> p.lineas().stream())` convierte todo en un solo `Stream<LineaPedido>`, que es lo que necesitamos para agrupar por producto.

Empate en producto más vendido
Leche tiene 6 unidades, Jamón 3 y Cereal 3. `max` devuelve uno de los empatados; el orden no debe considerarse una regla de negocio. Si el empate importa, define un desempate explícito.

Conceptos usados

`record`, `List.of`, `stream`, `mapToDouble`, `sum`, `flatMap`, `collect`, `Collectors.groupingBy`, `Collectors.summingDouble`, `Collectors.summingInt`, `Pedido::cliente`, `LineaPedido::subtotal`, `Map.Entry.comparingByValue`, `max`, `map`, `orElse`.

13. Validación de emails: sin regex y con regex

Implementar un validador de emails usando solo métodos de `String` (dos versiones) y luego refactorizarlo con una expresión regular. Se comparan las tres versiones con los mismos casos de prueba.

Archivos

Archivo	Qué contiene
Validar/src/validar/Sin_Regex/ValidarEmailV1.java	Validación paso a paso con `indexOf` y `substring`.
.../Sin_Regex/ValidarEmailV2.java	Validación compacta con una expresión booleana.
.../Regex/ValidarEmailRegex.java	`Pattern` y `Matcher`.
.../ValidacionInicial.java	`main` con tres casos: válido, inválido y sin arroba.

Paso a paso

1. V1: comprobar que no sea null ni vacío.
2. V1: exigir exactamente una `@` y que no esté al inicio.
3. V1: separar usuario y dominio; exigir un punto que no esté al inicio ni al final del dominio.
4. V1: rechazar espacios.
5. V2: resumir las mismas condiciones en una sola expresión booleana.
6. V3: compilar una vez el patrón y usar `matcher(email).matches()`.

Pseudocódigo / traza

```
V1:
    si email null o vacío: false
    arroba = indexOf("@")
    ultima = lastIndexOf("@")
    si arroba <= 0 o arroba != ultima: false
    usuario = substring(0, arroba)
    dominio = substring(arroba + 1)
    punto = indexOf(".") en dominio
    ultimoPunto = lastIndexOf(".") en dominio
    si punto <= 0 o ultimoPunto == dominio.length - 1: false
    si email contiene " ": false
    true

V3 regex:
    ^[\w._%+-]+@[\w.-]+\.[A-Za-z]{2,}$

Traza:
"usuario@ejemplo.com" -> true
"correo_invalido@.com" -> dominio empieza por "." -> false
"sin-arroba.com" -> no hay "@" -> false
```

ValidarEmailV1.java

```java
package validar.Sin_Regex;

public class ValidarEmailV1 {
    public static boolean esEmailValido(String email){
        if(email == null || email.isEmpty()){
            return false;
        }

        // Debe contener exactamente un arroba.
        int indiceArroba = email.indexOf("@");
        int ultimaArroba = email.lastIndexOf("@");

        if(indiceArroba <= 0 || indiceArroba != ultimaArroba){
            return false;
        }

        // Separar usuario y dominio.
        String usuario = email.substring(0, indiceArroba);
        String dominio = email.substring(indiceArroba + 1);

        // El dominio debe tener al menos un punto, no al inicio ni al final.
        int indicePunto = dominio.indexOf(".");
        int ultimoPunto = dominio.lastIndexOf(".");

        if(indicePunto <= 0 || ultimoPunto == dominio.length() - 1){
            return false;
        }

        // No debe tener espacios.
        if(email.contains(" ")){
            return false;
        }

        return true;
    }
}
```

ValidarEmailV2.java

```java
package validar.Sin_Regex;

public class ValidarEmailV2 {
    public static boolean esEmailValido(String email){
        if(email == null || email.isEmpty()){
            return false;
        }

        int at = email.indexOf('@');
        int punto = email.lastIndexOf('.');

        return at > 0
            && at < email.length() - 1
            && punto > at + 1
            && punto < email.length() - 1
            && email.indexOf(' ') == -1;
    }
}
```

ValidarEmailRegex.java

```java
package validar.Regex;

import java.util.regex.Pattern;

public class ValidarEmailRegex {
    private static final Pattern EMAIL = Pattern.compile(
        "^[\\w._%+-]+@[\\w.-]+\\.[A-Za-z]{2,}$"
    );

    public static boolean esEmailValido(String email){
        return email != null && EMAIL.matcher(email).matches();
    }
}
```

ValidacionInicial.java

```java
package validar;

import validar.Regex.ValidarEmailRegex;
import validar.Sin_Regex.ValidarEmailV1;
import validar.Sin_Regex.ValidarEmailV2;

public class ValidacionInicial {
    public static void main(String[] args){
        String test1 = "usuario@ejemplo.com";
        String test2 = "correo_invalido@.com";
        String test3 = "sin-arroba.com";

        System.out.println("== V1 sin regex ==");
        System.out.println(test1 + " -> " + ValidarEmailV1.esEmailValido(test1));
        System.out.println(test2 + " -> " + ValidarEmailV1.esEmailValido(test2));
        System.out.println(test3 + " -> " + ValidarEmailV1.esEmailValido(test3));

        System.out.println("== V2 sin regex ==");
        System.out.println(test1 + " -> " + ValidarEmailV2.esEmailValido(test1));
        System.out.println(test2 + " -> " + ValidarEmailV2.esEmailValido(test2));
        System.out.println(test3 + " -> " + ValidarEmailV2.esEmailValido(test3));

        System.out.println("== V3 con regex ==");
        System.out.println(test1 + " -> " + ValidarEmailRegex.esEmailValido(test1));
        System.out.println(test2 + " -> " + ValidarEmailRegex.esEmailValido(test2));
        System.out.println(test3 + " -> " + ValidarEmailRegex.esEmailValido(test3));
    }
}
```

Salida esperada

```
== V1 sin regex ==
usuario@ejemplo.com -> true
correo_invalido@.com -> false
sin-arroba.com -> false
== V2 sin regex ==
usuario@ejemplo.com -> true
correo_invalido@.com -> false
sin-arroba.com -> false
== V3 con regex ==
usuario@ejemplo.com -> true
correo_invalido@.com -> false
sin-arroba.com -> false
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
`main` solo probaba la versión con regex	Las clases V1 y V2 estaban importadas pero no se usaban.	Se añadieron los tres casos para comparar las tres implementaciones.

Error	Por qué fallaba	Corrección
Validación incompleta	V1 y V2 aceptan casos que la regex rechaza (por ejemplo, dominios con dos puntos seguidos).	Es intencional: son versiones didácticas. Para producción usa una regex más estricta o una librería.

Anatomía de la regex
`^` inicio, `[\\w._%+-]+` uno o más caracteres válidos para el usuario, `@`, `[\\w.-]+` dominio, `\\.` un punto literal, `[A-Za-z]{2,}` dos o más letras para la extensión, `$` final. `matches()` exige que toda la cadena cumpla el patrón.

Conceptos usados

`String.indexOf`, `lastIndexOf`, `substring`, `contains`, `isEmpty`, `Pattern`, `Matcher`, `matches`, `regex`, `boolean`.

14. Paginación manual (Pageable)

Dada una lista de 25 elementos, implementar `paginar(lista, page, size)` para devolver la sublista correcta y el total de páginas. Es la misma idea que `Pageable` de Spring Data: página, tamaño, contenido y total de páginas.

Archivos

Archivo	Qué contiene
Paginas/src/paginas/PaginacionManual/PaginacionManualV1.java	Devuelve solo la sublista (`List<T>`).
.../PaginacionManualV2.java	Devuelve un `Map<String, Object>` con contenido y metadatos.
Paginas/src/paginas/Paginar.java	`main` con 25 números, página 2, tamaño 10.

Paso a paso

1. Validar que la página sea al menos 1 y que el tamaño sea positivo.
2. Calcular el índice inicial: `(page - 1) * size` porque las páginas empiezan en 1.
3. Si el índice inicial es mayor o igual al total, devolver una lista vacía.
4. Calcular el índice final con `Math.min(inicio + size, total)` para no salir de la lista.
5. Devolver `subList(inicio, fin)` (V1) o un mapa con contenido y metadatos (V2).
6. Calcular `totalPaginas = ceil(total / size)`.

Pseudocódigo / traza

```
paginar(lista, page, size):
    si page < 1: page = 1
    si size < 1: size = 1

    total = lista.size()
    totalPaginas = ceil(total / size)
    from = min((page - 1) * size, total)
    to   = min(from + size, total)

    regresar {
        contenido: lista.subList(from, to),
        page, size,
        totalElementos: total,
        totalPaginas
    }

Traza con 25 elementos, page=2, size=10:
from = (2-1)*10 = 10
to   = min(10+10, 25) = 20
contenido = índices 10..19 = [11,12,...,20]
totalPaginas = ceil(25/10) = 3
```

PaginacionManualV1.java

```java
package paginas.PaginacionManual;

import java.util.ArrayList;
import java.util.List;

public class PaginacionManualV1 {

    // <T> permite paginar listas de cualquier tipo.
    public static <T> List<T> obtenerPagina(List<T> listaCompleta, int numeroPagina, int
        tamanoPagina){
        if(numeroPagina < 1){
            numeroPagina = 1;
        }

        int indiceInicial = (numeroPagina - 1) * tamanoPagina;

        if(indiceInicial >= listaCompleta.size()){
            return new ArrayList<>(); // página fuera de rango
        }

        int indiceFinal = Math.min(indiceInicial + tamanoPagina, listaCompleta.size());

        return listaCompleta.subList(indiceInicial, indiceFinal);
    }
}
```

PaginacionManualV2.java

```java
package paginas.PaginacionManual;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class PaginacionManualV2 {

    public static <T> Map<String, Object> paginar(List<T> lista, int page, int size){
        if(page < 1) page = 1;
        if(size < 1) size = 1;

        int total = lista.size();
        int totalPages = (int) Math.ceil((double) total / size);

        int from = Math.min((page - 1) * size, total);
        int to   = Math.min(from + size, total);

        Map<String, Object> respuesta = new LinkedHashMap<>();
        respuesta.put("contenido", lista.subList(from, to));
        respuesta.put("page", page);
        respuesta.put("size", size);
        respuesta.put("totalElementos", total);
        respuesta.put("totalPaginas", totalPages);
        return respuesta;
    }
}
```

Paginar.java

```java
package paginas;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

import paginas.PaginacionManual.PaginacionManualV1;
import paginas.PaginacionManual.PaginacionManualV2;

public class Paginar {
    public static void main(String[] args){
        List<Integer> numeros = new ArrayList<>();
        for(int i = 1; i <= 25; i++){
            numeros.add(i);
        }

        int paginaDeseada = 2;
        int elementosPorPagina = 10;

        List<Integer> resultado = PaginacionManualV1.obtenerPagina(numeros, paginaDeseada,
            elementosPorPagina);
        Map<String, Object> resultado2 = PaginacionManualV2.paginar(numeros, paginaDeseada,
            elementosPorPagina);

        System.out.println("Página " + paginaDeseada + ": " + resultado);
        System.out.println("Página " + paginaDeseada + ": " + resultado2);
    }
}
```

Salida esperada

```
Página 2: [11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
Página 2: {contenido=[11, 12, 13, 14, 15, 16, 17, 18, 19, 20], page=2, size=10,
    totalElementos=25, totalPaginas=3}
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
V1 y V2 usaban convenciones distintas	V1 trataba la página 2 como la segunda (índice 10), pero V2 usaba `page * size` (índice 20).	V2 ahora usa `(page - 1) * size`, igual que V1.
`Map.of` no garantiza orden	El mapa se imprimía en un orden arbitrario.	Se usó `LinkedHashMap` para conservar el orden de inserción.

subList es una vista

`lista.subList(from, to)` no copia los datos: devuelve una vista respaldada por la lista original. Si modificas la vista, se modifica la original. Si necesitas una copia independiente, usa `new ArrayList<>(lista.subList(from, to))`.

Conceptos usados

genéricos `<T>`, `List`, `ArrayList`, `subList`, `Math.min`, `Math.ceil`, `Map`, `LinkedHashMap`, `page/size`.

15. Cache con expiración (TTL)

Implementar una clase genérica `Cache<K, V>` que guarde valores con un tiempo de vida (TTL). Si se pide un valor expirado, debe devolver `null` y eliminarlo del mapa.

Archivos

Archivo	Qué contiene
CacheProblema/src/cacheProblema/CacheExperiracion.java	Clase genérica con `ConcurrentHashMap` y `Entry`.
CacheProblema/src/cacheProblema/Inicial.java	Demo: guardar, leer, esperar y comprobar expiración.

Paso a paso

1. Crear un `record Entry<V>(V valor, long expiraEn)` para guardar el valor y su instante de expiración.
2. Usar `ConcurrentHashMap<K, Entry<V>>` como almacenamiento thread-safe.
3. En `put`, calcular `System.currentTimeMillis() + ttlMillis` y guardar la entrada.
4. En `get`, buscar la entrada; si no existe, devolver `null`.
5. Si `ahora > expiraEn`, eliminar la clave y devolver `null`.
6. Si sigue vigente, devolver `e.valor()`.

Pseudocódigo / traza

```
put(key, value, ttl):
    expiraEn = ahora() + ttl
    store[key] = Entry(value, expiraEn)

get(key):
    e = store[key]
    si e == null: regresar null
    si ahora() > e.expiraEn:
        store.remove(key)
        regresar null
    regresar e.valor

Traza:
put("usuario:1", "Ana", 500)
get("usuario:1") -> Ana
esperar 700 ms
get("usuario:1") -> ahora > expiraEn -> remove -> null
```

CacheExperiracion.java

```java
package cacheProblema;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class CacheExperiracion<K, V> {

    // El record es privado: es un detalle interno de la cache.
    private record Entry<V>(V valor, long expiraEn) {}

    private final Map<K, Entry<V>> store = new ConcurrentHashMap<>();

    public void put(K key, V value, long ttlMillis){
        store.put(key, new Entry<>(value, System.currentTimeMillis() + ttlMillis));
    }

    public V get(K key){
        Entry<V> e = store.get(key);
        if(e == null) return null;

        if(System.currentTimeMillis() > e.expiraEn()){
            store.remove(key); // limpieza perezosa
            return null;
        }

        return e.valor();
    }
}
```

Inicial.java

```java
package cacheProblema;

public class Inicial {
    public static void main(String[] args) throws InterruptedException {
        CacheExperiracion<String, String> cache = new CacheExperiracion<>();

        cache.put("usuario:1", "Ana", 500); // vive 500 ms
        System.out.println("Recién guardado: " + cache.get("usuario:1"));

        Thread.sleep(700); // esperamos a que expire

        System.out.println("Después de 700 ms: " + cache.get("usuario:1"));
    }
}
```

Salida esperada

```
Recién guardado: Ana
Después de 700 ms: null
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
`Inicial` vacío	No había forma de ver la cache funcionando.	Se añadió una demo con TTL de 500 ms y una espera de 700 ms.
Nombre `ttMillis`	Era un typo de `ttlMillis` (time to live).	Se renombró el parámetro para que el propósito quede claro.
Limpieza solo al leer	Las entradas expiradas que nunca se leen permanecen en memoria.	Es una limpieza perezosa; en producción se añadiría un hilo o scheduler que recorra y elimine.

¿Por qué `ConcurrentHashMap`?
Una cache puede ser consultada por varios hilos al mismo tiempo. `HashMap` no es thread-safe; `ConcurrentHashMap` permite lecturas y escrituras concurrentes sin corromper la estructura.

Conceptos usados

`genéricos <K,V>`, `record anidado`, `ConcurrentHashMap`, `Map`, `System.currentTimeMillis`, `TTL`, `Thread.sleep`, `limpieza perezosa`.

16. Reintentos con backoff exponencial

Implementar `ejecutarConReintentos(Supplier<T>, intentos)`: ejecuta una tarea y, si lanza una excepción, espera un tiempo creciente y vuelve a intentarlo. Si se agotan los intentos, lanza la última excepción.

Archivos

Archivo	Qué contiene
RetryProblema/src/retryProblema/RetryBackoff.java	Bucle de intentos, `Supplier` y `Thread.sleep`.
RetryProblema/src/retryProblema/Inicial.java	Demo con una tarea que falla dos veces y funciona a la tercera.

Paso a paso

1. Recibir un `Supplier<T>` (la tarea) y el número máximo de intentos.
2. En cada intento, llamar a `tarea.get()` dentro de un `try`.
3. Si devuelve un valor, retornarlo inmediatamente.
4. Si lanza `RuntimeException`, guardarla como último error.
5. Si quedan intentos, esperar `2^i * 100` ms (100, 200, 400...).
6. Si se agotan, lanzar la última excepción guardada.

Pseudocódigo / traza

```
ejecutarConReintentos(tarea, intentos):
    ultimoError = null
    para i desde 0 hasta intentos - 1:
        intentar:
            regresar tarea.get()
        capturar RuntimeException e:
            ultimoError = e
            si i == intentos - 1: salir del bucle
            esperar 2^i * 100 ms
    lanzar ultimoError

Traza con 3 intentos:
i=0: falla -> espera 100 ms
i=1: falla -> espera 200 ms
i=2: funciona -> regresa "Conexión exitosa en el intento 3"
```

RetryBackoff.java

```java
package retryProblema;

import java.util.function.Supplier;

public class RetryBackoff {

    public static <T> T ejecutarConReintentos(Supplier<T> tarea, int intentos){
        RuntimeException ultimoError = null;

        for(int i = 0; i < intentos; i++){
            try{
                return tarea.get(); // si funciona, salimos
            } catch(RuntimeException e){
                ultimoError = e;

                if(i == intentos - 1) break; // ya no quedan intentos

                try{
                    // Backoff exponencial: 100 ms, 200 ms, 400 ms...
                    Thread.sleep((long) Math.pow(2, i) * 100);
                } catch(InterruptedException ie){
                    Thread.currentThread().interrupt();
                    throw new RuntimeException(ie);
                }
            }
        }

        throw ultimoError != null ? ultimoError : new RuntimeException("Falló sin excepción.");
    }
}
```

Inicial.java

```java
package retryProblema;

import java.util.concurrent.atomic.AtomicInteger;

public class Inicial {
    public static void main(String[] args){
        AtomicInteger contador = new AtomicInteger();

        String resultado = RetryBackoff.ejecutarConReintentos(() -> {
            int intento = contador.incrementAndGet();
            System.out.println("Intento " + intento);
            if(intento < 3){
                throw new RuntimeException("Fallo simulado " + intento);
            }
            return "Conexión exitosa en el intento " + intento;
        }, 5);

        System.out.println(resultado);
    }
}
```

Salida esperada

```
Intento 1
Intento 2
Intento 3
Conexión exitosa en el intento 3
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
`Inicial` vacío	No había tarea ni demostración.	Se añadió un `Supplier` que falla dos veces y un `AtomicInteger` para contar intentos.

Error	Por qué fallaba	Corrección
Espera innecesaria en el último intento	El código original dormía incluso después del último fallo.	Se añadió `if(i == intentos - 1) break;` antes del `sleep`.
Solo captura `RuntimeException`	Las excepciones comprobadas (`IOException`, etc.) no se reintentan.	Para una versión más general se puede capturar `Exception` o recibir una lista de excepciones reintentables.

¿Qué es backoff exponencial?
Cada reintento espera más que el anterior: 100, 200, 400, 800 ms. Así se evita saturar un servicio que está caído. En sistemas reales se añade un poco de aleatoriedad (jitter) para que varios clientes no reintenten exactamente al mismo tiempo.

Idempotencia
Reintentar solo es seguro si la operación se puede repetir sin efectos secundarios (por ejemplo, leer o actualizar de forma idempotente). Reintentar un cobro sin control puede duplicar el cargo.

Conceptos usados

`Supplier<T>`, `genéricos <T>`, `try/catch`, `RuntimeException`, `Thread.sleep`, `Math.pow`, `backoff exponencial`, `AtomicInteger`, `lambda`.

17. Parseo manual de JSON simple

Sin librerías, extraer el valor de una clave en un JSON plano como `{"nombre":"Ana","edad":25}`. Debe funcionar con valores entre comillas y con números, booleanos o `null`.

Archivos

Archivo	Qué contiene
ParseoProblema/src/parseoProblema/ParseoJson.java	Búsqueda de la clave, los dos puntos y el valor.
ParseoProblema/src/parseoProblema/Inicial.java	Pruebas con texto, número, booleano y clave inexistente.

Paso a paso

1. Buscar la clave entre comillas: `"nombre"`.
2. Buscar los dos puntos que van después de la clave.
3. Saltar espacios en blanco después de los dos puntos.
4. Si el valor empieza con `"`, buscar la siguiente comilla y extraer el texto.
5. Si no, leer hasta la coma o el cierre `}` y recortar espacios.
6. Si no se encuentra la clave o el valor, devolver `null`.

Pseudocódigo / traza

```
extraerValor(json, clave):
    patron = '"' + clave + '"'
    idx = json.indexOf(patron)
    si idx == -1: regresar null

    dosPuntos = json.indexOf(':', idx + patron.length)
    si dosPuntos == -1: regresar null

    inicio = dosPuntos + 1
    mientras json[inicio] sea espacio: inicio++

    si json[inicio] == '"':
        fin = json.indexOf('"', inicio + 1)
        regresar json.substring(inicio + 1, fin)

    fin = inicio
    mientras json[fin] != ',' y json[fin] != '}': fin++
    regresar json.substring(inicio, fin).trim()

Traza:
{"nombre":"Ana","edad":25,"activo":true}
clave "nombre" -> idx en "nombre", dosPuntos, inicio en '"' -> Ana
clave "edad"   -> valor no comillado -> lee hasta ',' -> 25
```

ParseoJson.java

```java
package parseoProblema;

public class ParseoJson {

    public static String extraerValor(String json, String clave){
        if(json == null || clave == null) return null;

        // Buscamos la clave CON sus comillas: "nombre".
        String patron = "\"" + clave + "\"";
        int idx = json.indexOf(patron);
        if(idx == -1) return null;

        // Los dos puntos que van después de la clave.
        int dosPuntos = json.indexOf(':', idx + patron.length());
        if(dosPuntos == -1) return null;

        // Saltamos espacios después de los dos puntos.
        int inicio = dosPuntos + 1;
        while(inicio < json.length() && Character.isWhitespace(json.charAt(inicio))){
            inicio++;
        }
        if(inicio >= json.length()) return null;

        // Caso 1: valor entre comillas.
        if(json.charAt(inicio) == '"'){
            int fin = json.indexOf('"', inicio + 1);
            if(fin == -1) return null;
            return json.substring(inicio + 1, fin);
        }

        // Caso 2: número, booleano o null.
        int fin = inicio;
        while(fin < json.length() && json.charAt(fin) != ',' && json.charAt(fin) != '}'){
            fin++;
        }
        return json.substring(inicio, fin).trim();
    }
}
```

Inicial.java

```java
package parseoProblema;

public class Inicial {
    public static void main(String[] args){
        String json = "{\"nombre\":\"Ana\",\"edad\":25,\"activo\":true}";

        System.out.println(ParseoJson.extraerValor(json, "nombre"));   // Ana
        System.out.println(ParseoJson.extraerValor(json, "edad"));     // 25
        System.out.println(ParseoJson.extraerValor(json, "activo"));   // true
        System.out.println(ParseoJson.extraerValor(json, "apellido")); // null
    }
}
```

Salida esperada

```
Ana
25
true
null
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
Patrón vacío	`String patron = "\"" + "\"";` producía `""`, así que `indexOf` encontraba la primera comilla y no la clave.	Se cambió a `String patron = "\"" + clave + "\"";`.

Error	Por qué fallaba	Corrección
Solo valores de texto	La versión original asumía que todo valor estaba entre comillas.	Se añadió el caso 2 para números, booleanos y `null`.
No se saltaban espacios	Un JSON con `"edad": 25` habría empezado a leer en el espacio.	Se añadió el bucle con `Character.isWhitespace`.

Limitaciones
Este parser no entiende comillas escapadas, objetos anidados, arreglos ni claves repetidas. Para JSON real usa Jackson, Gson o `jakarta.json`. El ejercicio sirve para entender `indexOf` y `substring`, no para producción.

Conceptos usados

`String.indexOf`, `substring`, `Character.isWhitespace`, `charAt`, `trim`, `JSON plano`, `null`.

18. Lista enlazada: operaciones y algoritmos

Implementar una lista enlazada simple con `NodoBasico`. Incluye agregar al final y al inicio, obtener por índice, eliminar por valor, imprimir, invertir, devolver como lista, detectar ciclos (Floyd), encontrar el nodo del medio, eliminar duplicados en lista ordenada y fusionar dos listas ordenadas.

Archivos

Archivo	Qué contiene
Nodos/src/nodos/NodoBasico.java	`valor` y `siguiente`.
Nodos/src/nodos/ListaEnlazada.java	Todas las operaciones.
Nodos/src/nodos/Nodos.java	`main` con trazas de cada operación.

Paso a paso

1. Un nodo guarda un valor y una referencia al siguiente; el último apunta a `null`.
2. Para agregar al final, recorrer hasta el último nodo y enlazar el nuevo.
3. Para agregar al inicio, el nuevo nodo apunta a la cabeza y la cabeza pasa a ser el nuevo.
4. Para eliminar, buscar el nodo anterior y saltar el nodo a borrar.
5. Para invertir, recorrer una vez cambiando cada `siguiente` hacia atrás.
6. Para detectar un ciclo, usar dos punteros: lento avanza 1, rápido avanza 2; si se encuentran, hay ciclo.
7. Para el nodo del medio, el mismo esquema: cuando el rápido llega al final, el lento está en el medio.
8. Para fusionar, comparar las cabezas y enlazar siempre el menor.

Pseudocódigo / traza

```
agregar(valor):
    nuevo = Nodo(valor)
    si cabeza == null: cabeza = nuevo
    si no:
        actual = cabeza
        mientras actual.siguiente != null: actual = actual.siguiente
        actual.siguiente = nuevo
    tamaño++

eliminar(valor):
    si cabeza == null: false
    si cabeza.valor == valor: cabeza = cabeza.siguiente; tamaño--; true
    actual = cabeza
    mientras actual.siguiente != null:
        si actual.siguiente.valor == valor:
            actual.siguiente = actual.siguiente.siguiente
            tamaño--; true
        actual = actual.siguiente
    false

invertir():
    anterior = null
    actual = cabeza
    mientras actual != null:
        siguiente = actual.siguiente
        actual.siguiente = anterior
        anterior = actual
        actual = siguiente
    cabeza = anterior

Traza:
agregar 10, 20, 30 -> 10 -> 20 -> 30
agregarAlInicio 5  -> 5 -> 10 -> 20 -> 30
obtener(2)         -> 20
eliminar(20)       -> 5 -> 10 -> 30
invertir()         -> 30 -> 10 -> 5
```

NodoBasico.java

```
package nodos;

public class NodoBasico {
    int valor;
    NodoBasico siguiente;

    NodoBasico(int valor){
        this.valor = valor;
        this.siguiente = null;
    }
}
```

ListaEnlazada.java (parte 1: operaciones básicas)

```
package nodos;

import java.util.ArrayList;
import java.util.List;

public class ListaEnlazada {
    private NodoBasico cabeza;
    private int tamaño;

    public ListaEnlazada(){
        this.cabeza = null;
        this.tamaño = 0;
    }

    public void agregar(int valor){
        NodoBasico nuevo = new NodoBasico(valor);

        if(cabeza == null){
            cabeza = nuevo;
        } else {
            NodoBasico actual = cabeza;
            while(actual.siguiente != null){
                actual = actual.siguiente;
            }
            actual.siguiente = nuevo;
        }
        tamaño++;
    }

    public void agregarAlInicio(int valor){
        NodoBasico nuevo = new NodoBasico(valor);
        nuevo.siguiente = cabeza;
        cabeza = nuevo;
        tamaño++;
    }

    public int obtener(int indice){
        if(indice < 0 || indice >= tamaño){
```

```
            throw new IndexOutOfBoundsException("Índice inválido: " + indice);
        }

        NodoBasico actual = cabeza;
        for(int i = 0; i < indice; i++){
            actual = actual.siguiente;
        }
        return actual.valor;
    }

    public boolean eliminar(int valor){
        if(cabeza == null) return false;

        if(cabeza.valor == valor){
            cabeza = cabeza.siguiente;
            tamaño--;
            return true;
        }

        NodoBasico actual = cabeza;
        while(actual.siguiente != null){
            if(actual.siguiente.valor == valor){
                actual.siguiente = actual.siguiente.siguiente;
                tamaño--;
                return true;
            }
            actual = actual.siguiente;
        }
        return false;
    }

    public void imprimir(){
        NodoBasico actual = cabeza;
        while(actual != null){
            System.out.print(actual.valor + " -> ");
            actual = actual.siguiente;
        }
        System.out.print("null\n");
```

```
    }

    public int getTamaño(){
        return tamaño;
    }

    public boolean estaVacia(){
        return cabeza == null;
    }
```

ListaEnlazada.java (parte 2: algoritmos)

```
    // Invertir in-place: no crea otra lista.
    public void invertir(){
        NodoBasico anterior = null;
        NodoBasico actual = cabeza;

        while(actual != null){
            NodoBasico siguiente = actual.siguiente; // guardar el resto
            actual.siguiente = anterior;             // dar la vuelta al enlace
            anterior = actual;                       // avanzar anterior
            actual = siguiente;                      // avanzar actual
        }
        cabeza = anterior;
    }

    public List<Integer> aLista(){
        List<Integer> resultado = new ArrayList<>();
        NodoBasico actual = cabeza;

        while(actual != null){
            resultado.add(actual.valor);
            actual = actual.siguiente;
        }
        return resultado;
    }

    // Floyd: si hay ciclo, lento y rápido se encuentran.
    public boolean tieneCiclo(){
        NodoBasico lento = cabeza;
        NodoBasico rapido = cabeza;

        while(rapido != null && rapido.siguiente != null){
            lento = lento.siguiente;
            rapido = rapido.siguiente.siguiente;
            if(lento == rapido) return true;
        }
        return false;
    }

```

```
    public int nodoMedio(){
        if(cabeza == null) throw new IllegalStateException("Lista vacía");

        NodoBasico lento = cabeza;
        NodoBasico rapido = cabeza;

        while(rapido != null && rapido.siguiente != null){
            lento = lento.siguiente;
            rapido = rapido.siguiente.siguiente;
        }
        return lento.valor;
    }

    // Solo para listas ordenadas.
    public void eliminarDuplicados(){
        NodoBasico actual = cabeza;

        while(actual != null && actual.siguiente != null){
            if(actual.valor == actual.siguiente.valor){
                actual.siguiente = actual.siguiente.siguiente;
                tamaño--;
            } else {
                actual = actual.siguiente;
            }
        }
    }

    // Fusiona dos listas ordenadas en una nueva.
    public ListaEnlazada fusionar(ListaEnlazada otra){
        ListaEnlazada resultado = new ListaEnlazada();
        NodoBasico a = this.cabeza;
        NodoBasico b = otra.cabeza;

        while(a != null && b != null){
            if(a.valor <= b.valor){
                resultado.agregar(a.valor);
                a = a.siguiente;
            } else {
                resultado.agregar(b.valor);
                b = b.siguiente;
            }
        }

        while(a != null){
            resultado.agregar(a.valor);
            a = a.siguiente;
        }

        while(b != null){
            resultado.agregar(b.valor);
            b = b.siguiente;
        }

        return resultado;
    }
}
```

Nodos.java (main)

```
package nodos;

public class Nodos {
    public static void main(String[] args) {
        ListaEnlazada lista = new ListaEnlazada();

        lista.agregar(10);
        lista.agregar(20);
        lista.agregar(30);
        lista.agregarAlInicio(5);
        lista.imprimir(); // 5 -> 10 -> 20 -> 30 -> null

        System.out.println("Valor en índice 2: " + lista.obtener(2)); // 20
        System.out.println("Tamaño: " + lista.getTamaño());

        lista.eliminar(20);
        lista.imprimir(); // 5 -> 10 -> 30 -> null

        lista.invertir();
        System.out.print("Invertida: ");
        lista.imprimir(); // 30 -> 10 -> 5 -> null

        System.out.println("Como lista: " + lista.aLista());
        System.out.println("Nodo del medio: " + lista.nodoMedio());
        System.out.println("¿Tiene ciclo? " + lista.tieneCiclo());

        ListaEnlazada conDuplicados = new ListaEnlazada();
        conDuplicados.agregar(1);
        conDuplicados.agregar(1);
        conDuplicados.agregar(2);
        conDuplicados.agregar(3);
        conDuplicados.agregar(3);
        conDuplicados.eliminarDuplicados();
        System.out.print("Sin duplicados: ");
        conDuplicados.imprimir(); // 1 -> 2 -> 3 -> null

        ListaEnlazada a = new ListaEnlazada();
        a.agregar(1); a.agregar(4); a.agregar(7);
```

```

        ListaEnlazada b = new ListaEnlazada();
        b.agregar(2); b.agregar(3); b.agregar(8);

        ListaEnlazada fusionada = a.fusionar(b);
        System.out.print("Fusionada: ");
        fusionada.imprimir(); // 1 -> 2 -> 3 -> 4 -> 7 -> 8 -> null
    }
}
```

Salida esperada

```
5 -> 10 -> 20 -> 30 -> null
Valor en índice 2: 20
Tamaño: 4
5 -> 10 -> 30 -> null
Invertida: 30 -> 10 -> 5 -> null
Como lista: [30, 10, 5]
Nodo del medio: 10
¿Tiene ciclo? false
Sin duplicados: 1 -> 2 -> 3 -> null
Fusionada: 1 -> 2 -> 3 -> 4 -> 7 -> 8 -> null
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
`invertir` no actualizaba la cabeza	El método recibía un `NodoBasico` y devolvía la nueva cabeza, pero el `main` no la asignaba.	Se añadió `invertir()` sin parámetros que actualiza `cabeza = anterior`.

Error	Por qué fallaba	Corrección
Métodos sin implementar	Las secciones de ciclo, medio, duplicados y fusión estaban vacías.	Se implementaron con punteros y con el algoritmo de Floyd.
`main` con la inversión comentada	No se probaba el método.	Se descomentó y se añadieron pruebas de las demás operaciones.

La clave de invertir
En cada vuelta hay que guardar `siguiente` antes de cambiar `actual.siguiente`; si no, se pierde el resto de la lista. El orden es: guardar, invertir, avanzar anterior, avanzar actual.

Conceptos usados

`nodo`, `referencias`, `while`, `invertir in-place`, `Floyd (tortuga y liebre)`, `nodo medio`, `eliminar duplicados`, `fusionar listas`, `complejidad O(n)`.

19. Mini Spring: anotación y contenedor de beans

Crear una anotación `@MiServicio` y un contenedor que instancie automáticamente las clases anotadas. Primero crea las que no tienen dependencias y después las que reciben otras por constructor. Es una versión mínima del contenedor de Spring.

Archivos

Archivo	Qué contiene
MiniSpring/src/miniSpring/Interfaces/MiServicio.java	Anotación con `@Retention(RUNTIME)` y `@Target(TYPE)`.
.../Implementaciones/MiniContenedor.java	Mapa de beans, escaneo y reflexión.
.../Service/EmailService.java	Bean sin dependencias.
.../Service/UserService.java	Bean que depende de `EmailService`.
.../Inicial.java	`main` que escanea y pide el bean.

Paso a paso

1. Definir `@MiServicio` para que esté disponible en tiempo de ejecución.
2. El contenedor guarda `Map<Class<?>, Object> beans`.
3. Primera pasada: para cada clase anotada con constructor sin parámetros, crear la instancia y guardarla.
4. Segunda pasada: para las que faltan, obtener su constructor, mirar los tipos de parámetros y buscar cada uno en el mapa.
5. Crear la instancia con `ctor.newInstance(args)` y guardarla.
6. `getBean(Class<T>)` busca, comprueba que exista y hace `tipo.cast(...)`.
7. El `main` escanea, pide `UserService` y llama a `registrar()`.

Pseudocódigo / traza

```
@MiServicio
class EmailService { enviar() }

@MiServicio
class UserService {
    EmailService emailService
    UserService(EmailService e) { emailService = e }
    registrar() { imprimir "Usuario registrado"; emailService.enviar() }
}

contenedor.escanear():
    classes = [EmailService, UserService]

    // primera pasada
    para c en classes:
        si c tiene @MiServicio y constructor sin parámetros:
            beans[c] = new c()

    // segunda pasada
    para c en classes:
        si c no tiene @MiServicio o ya está en beans: continuar
        ctor = c.getConstructors()[0]
        args = para cada tipo de parámetro: beans[tipo]
        beans[c] = ctor.newInstance(args)

main:
    ctx.escanear(...)
    userService = ctx.getBean(UserService.class)
    userService.registrar()
```

MiServicio.java

```java
package miniSpring.Interfaces;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME) // visible en tiempo de ejecución
@Target(ElementType.TYPE)           // solo se puede poner en clases
public @interface MiServicio {
}
```

MiniContenedor.java

```java
package miniSpring.Implementaciones;

import java.lang.reflect.Constructor;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import miniSpring.Interfaces.MiServicio;
import miniSpring.Service.EmailService;
import miniSpring.Service.UserService;

public class MiniContenedor {
    private final Map<Class<?>, Object> beans = new HashMap<>();

    public void escanear(String paquete) throws Exception{
        // Versión simplificada: la lista está escrita a mano.
        List<Class<?>> classes = List.of(EmailService.class, UserService.class);

        // PRIMERA PASADA: beans sin dependencias.
        for(Class<?> c : classes){
            if(!c.isAnnotationPresent(MiServicio.class)) continue;

            Constructor<?>[] constructores = c.getConstructors();

            if(constructores.length > 0 && constructores[0].getParameterCount() == 0){
                beans.put(c, c.getDeclaredConstructor().newInstance());
            }
        }

        // SEGUNDA PASADA: beans con dependencias.
        for(Class<?> c : classes){
            if(!c.isAnnotationPresent(MiServicio.class) || beans.containsKey(c)) continue;

            Constructor<?> ctor = c.getConstructors()[0];

            Object[] args = Arrays.stream(ctor.getParameterTypes())
                                  .map(beans::get)
```

```java
                                  .toArray();

            beans.put(c, ctor.newInstance(args));
        }
    }

    public <T> T getBean(Class<T> tipo){
        Object bean = beans.get(tipo);
        if(bean == null){
            throw new IllegalArgumentException("No hay bean para " + tipo.getName());
        }
        return tipo.cast(bean);
    }
}
```

EmailService.java y UserService.java

```
 1 | package miniSpring.Service;
 2 |
 3 | import miniSpring.Interfaces.MiServicio;
 4 |
 5 | @MiServicio
 6 | public class EmailService {
 7 |     public void enviar(){
 8 |         System.out.println("Email enviado");
 9 |     }
10 | }
11 |
12 | // ------------------------------------------------------------
13 |
14 | package miniSpring.Service;
15 |
16 | import miniSpring.Interfaces.MiServicio;
17 |
18 | @MiServicio
19 | public class UserService {
20 |     private final EmailService emailService;
21 |
22 |     public UserService(EmailService emailService){
23 |         this.emailService = emailService;
24 |     }
25 |
26 |     public void registrar(){
27 |         System.out.println("Usuario registrado");
28 |         emailService.enviar();
29 |     }
30 | }
```

Inicial.java

```
 1 | package miniSpring;
 2 |
 3 | import miniSpring.Implementaciones.MiniContenedor;
 4 | import miniSpring.Service.UserService;
 5 |
 6 | public class Inicial {
 7 |     public static void main(String[] args) throws Exception {
 8 |         MiniContenedor ctx = new MiniContenedor();
 9 |         ctx.escanear("miniSpring.Service");
10 |
11 |         UserService userService = ctx.getBean(UserService.class);
12 |         userService.registrar();
13 |         // Usuario registrado
14 |         // Email enviado
15 |     }
16 | }
```

Salida esperada

```
Usuario registrado
Email enviado
```

Errores encontrados y cómo se corrigieron

Error	Por qué fallaba	Corrección
`getConstructor()[0]`	`getConstructor()` devuelve **un** `Constructor`, no un arreglo; no se puede indexar.	Se usó `getConstructors()[0]` (plural) para obtener el arreglo y tomar el primero.
Faltaban imports	`MiniContenedor` usaba `EmailService` y `UserService` sin importarlos.	Se añadieron los imports de `miniSpring.Service`.

Error	Por qué fallaba	Corrección
Clases package-private	`class EmailService` y `class UserService` no eran públicas; `MiniContenedor` está en otro paquete y no podía instanciarlas.	Se declararon `public class` y constructores públicos.
`getBean` devolvía null	Si el bean no existía, `tipo.cast(null)` devolvía null y fallaba más tarde.	Se comprueba `bean == null` y se lanza un mensaje claro.
Paquete de ejemplo inexistente	`ctx.escanear("com.ejemplo")` no correspondía a las clases reales.	Se pasó `"miniSpring.Service"` (el parámetro es informativo en esta versión simplificada).

Qué hace Spring de verdad
Spring escanea el classpath, detecta anotaciones, resuelve dependencias, maneja scopes (singleton, prototype), aplica proxies para transacciones y seguridad, y publica eventos. Este contenedor solo reproduce la idea central: registrar objetos y resolver sus dependencias.

Orden de instanciación
La primera pasada crea los beans sin dependencias. Si un bean depende de otro que también tiene dependencias, haría falta resolverlo recursivamente o repetir pasadas hasta que no queden pendientes. Esa es una de las complicaciones que Spring resuelve por ti.

Conceptos usados

`anotación`, `@Retention`, `@Target`, `reflexión`, `Class<?>`, `Constructor<?>`, `getConstructors`, `getParameterCount`, `getParameterTypes`, `newInstance`, `Map<Class<?>, Object>`, `getBean`, `cast`, `beans`.

Parte III — Errores corregidos

Estos son los errores que impedían compilar o que producían resultados incorrectos. La columna "causa" explica el porqué, no solo el síntoma; la columna "corrección" muestra el cambio aplicado.

Proyecto	Error	Causa	Corrección
Colecciones	Recursión infinita	Se llamaba a `medianSorted(nums1, nums2)` con los mismos argumentos	`medianSorted(nums2, nums1)`
Colecciones	Partición Y incorrecta	Faltaba restar `partitionX`	`(m + n + 1) / 2 - partitionX`
Colecciones	`Scanner` cerrado en el bucle	Cerrar el Scanner cierra `System.in`	Un Scanner en `main`, cerrado al final
Colecciones	Opciones del menú intercambiadas	`case 1` llamaba a la opción 2	Se alinearon los `case`
Colecciones	Datos desordenados	La mediana asume arreglos ordenados	`nums2 = {3, 6, 9, 10}`
ApiConsola	`while` en el cuerpo de la clase	Java no permite sentencias sueltas	Método `iniciar()`
ApiConsola	`orElseThrow ->`	Lambda sin paréntesis	`orElseThrow(() -> ...)`
ApiConsola	`Tarea` no importada	Record anidado en otro paquete	`import ...ApiModel.Tarea`
ApiConsola	`save` no actualizaba	`db.put` dentro del `if`	`db.put` siempre
ApiConsola	`next()` cortaba el título	Lee solo una palabra	`nextLine()`
MiniSpring	`getConstructor()[0]`	`getConstructor()` no devuelve arreglo	`getConstructors()[0]`
MiniSpring	Faltaban imports	No se importaban los servicios	Imports añadidos
MiniSpring	Clases no públicas	Otro paquete no puede instanciarlas	`public class`
MiniSpring	`getBean` devolvía null	No se comprobaba el bean	`IllegalArgumentException`
Pedidos	`pedidos` inexistente	No había campo ni constructor	Campo `List<Pedido>` + constructor
Pedidos	Records inaccesibles	Visibilidad de paquete	Archivos públicos separados
ParseoProblema	Patrón vacío	`"\"" + "\""` producía `""`	Incluir la clave en el patrón
Promedio	Categoría equivocada	Filtraba `Alimentos`	`Electronica` con `equalsIgnoreCase`
Paginas	V1 y V2 con bases distintas	V2 usaba `page * size`	`(page - 1) * size`
Mapas	Resultado esperado no coincidía	La frase tenía `spring` una sola vez	Se agregó la segunda aparición
Nodos	`invertir` no actualizaba la cabeza	Recibía y devolvía un nodo	`invertir()` sin parámetros
Nodos	Métodos vacíos	TODOs sin implementar	Floyd, medio, duplicados, fusión
RetryProblema	Espera en el último intento	`sleep` después del último fallo	`break` antes del `sleep`
Varios	`Inicial` vacío	No había `main`	Se añadió una demo ejecutable

La mayoría de estos errores no eran de lógica sino de **visibilidad**, **sintaxis** o **estado compartido** (el Scanner, el mapa que no se actualizaba). Cuando algo no compile, revisa primero los imports y la visibilidad; cuando compile pero dé un resultado raro, revisa el estado y los índices.

Parte IV — Cómo compilar y ejecutar

No hay Maven ni Gradle: son proyectos de Java "puros". Puedes ejecutarlos desde VS Code o desde la terminal. En ambos casos la clave es compilar todos los `.java` del proyecto y luego ejecutar la clase que tiene `main` usando su nombre completamente calificado (paquete + clase).

Desde VS Code

El archivo `.vscode/settings.json` ya declara las 19 carpetas `src` y el directorio de salida `bin`. Abre el archivo que tiene `main`, pulsa el botón de ejecutar (Run) y VS Code compila y lanza esa clase. Si aparece un error de paquete, comprueba que la ruta del archivo coincida con la declaración `package`.

Desde PowerShell, todo el repositorio

```
# Situarse en la raíz del repositorio
cd "C:\Users\Iracar Tech\Documents\Personal Iracar Technologies\Practicar logica"

# Compilar todos los .java en una carpeta de salida
$out = "out"
New-Item -ItemType Directory -Force $out | Out-Null
$files = @(Get-ChildItem -Recurse -File -Filter *.java | ForEach-Object { $_.FullName })
javac -encoding UTF-8 -d $out $files

# Ejecutar una clase (paquete + nombre)
java -cp $out sumardosnumeros.SumarDosNumeros
java -cp $out listas.Listas
java -cp $out pedidos.Inicial
```

Desde PowerShell, un solo proyecto

```
cd "C:\Users\Iracar Tech\Documents\Personal Iracar Technologies\Practicar logica"

# Ejemplo con Listas
$out = "out-listas"
New-Item -ItemType Directory -Force $out | Out-Null
$files = @(Get-ChildItem "Listas\src" -Recurse -File -Filter *.java | ForEach-Object {
    $_.FullName })
javac -encoding UTF-8 -d $out $files
java -cp $out listas.Listas
```

Clases con main

Proyecto	Clase a ejecutar	¿Pide datos?
SumarDosNumeros	`sumardosnumeros.SumarDosNumeros`	No
Operaciones	`operaciones.Operaciones`	Sí (grados)
Colecciones	`colecciones.Colecciones`	Sí (menú)
Listas	`listas.Listas`	No
Mapas	`mapas.Mapas`	No
Colectores	`colectores.Colectores`	No
Promedio	`promedio.Inicial`	No
Descuento	`descuento.DescuentoMain`	No
InyeccionDeDependencias	`inyeccionDeDependencias.InyeccionInicial`	No
Repositorio	`repositorio.RepositorioInicial`	No

Proyecto	Clase a ejecutar	¿Pide datos?
ApiConsola	`apiConsola.Inicial`	Sí (menú)
Pedidos	`pedidos.Inicial`	No
Validar	`validar.ValidacionInicial`	No
Paginas	`paginas.Paginar`	No
CacheProblema	`cacheProblema.Inicial`	No (espera 700 ms)
RetryProblema	`retryProblema.Inicial`	No (espera backoff)
ParseoProblema	`parseoProblema.Inicial`	No
Nodos	`nodos.Nodos`	No
MiniSpring	`miniSpring.Inicial`	No

Error típico: "could not find or load main class"
Casi siempre significa que el nombre de la clase no incluye el paquete o que la carpeta de salida no es la correcta. Usa el nombre completo, por ejemplo `java -cp out pedidos.Inicial`, y no `java -cp out Inicial`.

Parte V — Checklist y siguientes pasos

Ya tienes los 19 proyectos compilando y con una demo ejecutable. La parte más importante no es que funcionen, sino que puedas explicar por qué funcionan. Usa esta lista para autoevaluarte.

Checklist por conceptos

- Puedo explicar la diferencia entre `List`, `ArrayList`, `Set` y `HashSet`.
- Sé cuándo usar `HashMap` y por qué `ConcurrentHashMap` es thread-safe.
- Entiendo qué hace `stream()` y por qué las operaciones intermedias son perezosas.
- Puedo escribir un pipeline con `filter`, `map`, `sorted` y `collect` sin mirar apuntes.
- Sé la diferencia entre `map` y `flatMap`.
- Entiendo `Collectors.groupingBy` con y sin colector descendente.
- Puedo explicar `Pedido::cliente` y escribir su lambda equivalente.
- Sé qué es un `Supplier<T>` y por qué `orElseThrow` lo usa.
- Entiendo qué genera un `record` y por qué no tiene setters.
- Puedo leer `Class<?> c : classes` y explicar qué hace `c.getConstructors()`.
- Sé qué es un `Optional` y por qué es mejor que devolver `null`.
- Entiendo la diferencia entre `Collections.frequency` y `Collectors`.
- Puedo explicar por qué cerrar un `Scanner` cierra `System.in`.
- Sé qué es un TTL y cómo se comprueba en una cache.
- Entiendo el algoritmo de Floyd para detectar ciclos.

Errores que conviene evitar

- No cierres `System.in` dentro de un bucle ni en métodos que reciben un `Scanner` compartido.
- No uses `==` para comparar `String`; usa `equals` o `equalsIgnoreCase`.
- No devuelvas el `Map` o la `List` interna directamente; devuelve una copia.
- No uses `next()` si esperas una frase; usa `nextLine()` y consume el salto pendiente.
- No olvides que `List.of` y `Map.of` son inmutables.
- No asumas que `HashMap` o `HashSet` conservan el orden.
- No uses `Collections.frequency` dentro de un `filter` para listas grandes: es $O(n^2)$.
- No captures `Exception` sin distinguir si el reintento tiene sentido.
- No hagas `sleep` después del último intento.
- No confundas visibilidad de paquete con `public`: si otra capa lo usa, debe ser público.

Siguientes ejercicios recomendados

Para seguir practicando, intenta ampliar los proyectos existentes en lugar de empezar otros desde cero. Algunas ideas concretas:

- **Repositorio:** añade `findByEmail`, `existsById` y paginación con `Pageable` simulado.
- **ApiConsola:** añade prioridad, fecha de creación y un comando para buscar por título.
- **Pedidos:** añade descuentos por cliente y un informe de ventas por mes.
- **Cache:** añade un método `remove`, un tamaño máximo y limpieza programada.
- **Retry:** añade jitter y una lista de excepciones reintentables.

- **Nodos:** añade `detectarYCortarCiclo`, `fusionar` sin crear una lista nueva y una lista doblemente enlazada.
- **MiniSpring:** añade `@MiComponente`, inyección por setter y detección de dependencias circulares.
- **Validar:** añade validación de teléfonos y contraseñas con requisitos.

Cómo estudiar con este PDF

Lee primero el objetivo y el pseudocódigo; intenta escribirlo tú antes de mirar el código. Después compara tu versión con la del PDF y ejecuta el `main` para ver la salida. Si algo no cuadra, vuelve a la Parte I y busca el concepto. La repetición espaciada funciona mejor que leer todo de una vez: haz un ejercicio, explica en voz alta qué hace cada línea y vuelve al día siguiente.

Último consejo
No memorices los streams. Aprende a identificar la fuente, la transformación y el resultado. Cuando tengas claro qué entra y qué sale en cada etapa, el código se escribe casi solo.